

دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق

آزمایشگاه رباتیک و بینایی ماشین

استاد: دکتر حمیدرضا تقی راد

استاد: دکتر محمدجواد احمدی

گزارش سری اول

گروه آزادی

نام و نام خانوادگی	محمدامین محمدیون شبستری
شماره دانشجویی	۴۰۱۲۲۵۰۳
نام و نام خانوادگی	محمدسبحان سخایی
شماره دانشجویی	۴۰۱۱۹۲۵۳
نام و نام خانوادگی	هستی محمدی (انجام فعالیت به صورت امتیازی)
شماره دانشجویی	۴۰۲۲۱۲۴۳
تاریخ	اردیبهشت ۱۴۰۵



فهرست مطالب

۴	بخش C تمرین سری دوم: شبیه‌سازی و اعتبارسنجی نهایی در PyBullet	۱
۴	بخش اول: نوشتن مدل URDF ربات	۱.۱
۵	تسک ۲.۱ و ۲.۳: یکپارچه‌سازی پایپ‌لاین، راه‌اندازی سلول کاری و اعتبارسنجی فرآیند بازرسی	۲.۱
۸	تسک ۳.۱: برنامه‌ریزی مسیر، شبیه‌سازی دینامیک حرکت و تولید انیمیشن فرآیند بازرسی	۳.۱
۱۱	تسک ۴.۱: گزارش نهایی یکپارچه و داشبورد مانیتورینگ جامع سیستم بازرسی	۴.۱
۱۴	چالش اصلی و مهندسی سیستم: پدیده برخورد با خود (Self-Collision) و اصلاح ساختاری مفصل	۵.۱
۱۶	کد نهایی	۶.۱
۱۶	کد ۱.۶.۱	



فهرست تصاویر

۱	نمودار خطای اعتبارسنجی سینماتیک مستقیم بر حسب میلی‌متر (ارزیابی عملکرد مدل URDF در برابر روابط جبری)	۷
۲	تحلیل گام‌به‌گام فریم‌های رندر شده از فرآیند بازرسی خودکار صلب در محیط PyBullet	۱۱
۳	گزارش نهایی یکپارچه سیستم بازرسی خودکار ربات PRR (خروجی تجمیعی پایپ‌لاین بینایی، فضای کاری و تحلیل خطا)	۱۲
۴	خطای فیزیکی مکانیزم در پیاده‌سازی اولیه: رخ دادن پدیده برخورد با خود (Self-Collision) و تداخل صلب لینک‌های دوم و سوم به دلیل عدم اعمال محدودیت زاویه‌ای فیزیکی	۱۴



فهرست جداول

۱	نتایج اعتبارسنجی مقاطع سیستم مختصات ابزار ربات در دو فضای فیزیکی و تحلیلی	۶
---	---	---



۱ بخش C تمرین سری دوم: شبیه‌سازی و اعتبارسنجی نهایی در PyBullet

۱.۱ بخش اول: نوشتن مدل URDF ربات

در این بخش از پروژه، هدف ایجاد یک مدل استاندارد از ربات برای استفاده در محیط شبیه‌ساز فیزیکی PyBullet است. برای این منظور از فرمت استاندارد URDF استفاده شده است.

مدل URDF چیست؟

فرمت URDF (مخفف Unified Robot Description Format) یک استاندارد توصیف سخت‌افزاری مبتنی بر زبان نشانه‌گذاری XML است که در سیستم‌عامل ربات‌ها (ROS) و موتورهای شبیه‌ساز فیزیکی (مانند PyBullet و Gazebo) برای توصیف دقیق ساختار فیزیکی، سینماتیکی و دینامیکی ربات‌ها استفاده می‌شود. در این فایل، ربات به عنوان مجموعه‌ای از بدنه‌های صلب (به نام Link) که توسط مفاصل (به نام Joint) به یکدیگر متصل شده‌اند، توصیف می‌گردد.

هر پیوند (Link) در این ساختار به طور کلی شامل سه بخش اصلی است:

- **Visual:** ویژگی‌های ظاهری و بصری ربات (مانند اشکال هندسی پایه اعم از مکعب و استوانه، ابعاد و رنگ) را برای رندرینگ گرافیکی تعیین می‌کند.

- **Collision:** هندسه‌ی برخورد ربات را مشخص می‌کند. موتور فیزیک از این هندسه برای محاسبه تداخل‌ها (Intersections) و برخوردهای فیزیکی با محیط استفاده می‌کند.

- **Inertial:** پارامترهای دینامیکی مانند جرم (Mass) و ماتریس ممان اینرسی (Inertia Tensor) را در بر می‌گیرد که برای شبیه‌سازی دینامیک و اعمال گشتاورها ضروری است.

مفاصل یا Joint‌ها نیز وظیفه تعریف اتصالات را بر عهده دارند. در هر مفصل پارامترهایی نظیر نوع حرکت (کشویی، دورانی، ثابت و غیره)، محور دوران یا جابجایی (Axis)، محدودیت‌های حرکتی (Limits) و موقعیت نسبی سیستم مختصات دو لینک نسبت به یکدیگر (Origin) تعریف می‌شود.

ساختار ربات PRR در این پروژه

بر اساس الزامات مسئله، بازوی مورد بررسی یک ربات بازرس با ساختار PRR (یک مفصل کشویی و دو مفصل دورانی) است. برای اینکه پروژه کاملاً مستقل (Standalone) باشد و نیازی به آپلود فایل‌های جانبی نباشد، کدهای XML مربوط به این ساختار مستقیماً به عنوان یک متغیر متنی طولانی (String) در اسکریپت پایتون نوشته شده است. این متن در زمان اجرا در یک فایل موقت (Temp) در سیستم‌عامل ذخیره می‌شود تا PyBullet آن را بارگذاری کند.

ساختار سلسله‌مراتبی این ربات به شرح زیر مدل‌سازی شده است:

۱. پایه ربات (**base_link**): این بخش به عنوان ریشه (Root) زنجیره سینماتیکی ربات و به صورت ثابت روی میز کار تعریف شده است. از نظر طراحی مهندسی، برای جلوگیری از مسدود شدن دید دوربین (ایجاد نقطه کور روی دیسک‌های زیر ربات)، شعاع استوانه پایه بسیار باریک (با شعاع 0.04 متر) در نظر گرفته شده است.



۲. لینک اول ($link1$) و مفصل کشویی ($j1_prismatic$): این لینک از طریق یک مفصل منشوری یا کشویی ($Prismatic$) به پایه متصل است و در امتداد محور Z به سمت بالا و پایین حرکت می‌کند. محدودیت حرکتی ($Limit$) این مفصل بین صفر تا 0.5 متر تنظیم شده است تا ربات بتواند ارتفاع خود را با سطح کار تطبیق دهد.

۳. لینک دوم ($link2$) و مفصل دورانی اول ($j2_revolute$): بازوی افقی اول (شانه ربات) که طول آن برابر با پارامتر L_2 است. این لینک با یک مفصل دورانی حول محور Z به لینک قبلی متصل می‌شود. برای جلوگیری از چرخش‌های بی‌نهایت و شبیه‌سازی دقیق شرایط صنعتی، نوع مفصل از حالت آزاد ($Continuous$) به حالت محدود ($Revolute$) تغییر یافته و دامنه حرکتی آن بین $-\pi$ تا $+\pi$ رادیان محدود شده است.

۴. لینک سوم ($link3$) و مفصل دورانی دوم ($j3_revolute$): بازوی افقی دوم (آرنج ربات) به طول L_3 . این مفصل نیز از نوع $Revolute$ است. در طراحی بهینه‌ی این مدل، برای جلوگیری از مشکل برخورد بازو با خودش ($Self-Collision$) — یعنی حالتی که بازو 180° درجه چرخیده و از داخل بدنه قبلی عبور کند — محدودیت زاویه‌ای اکیدی (از -3.05 تا $+3.05$ رادیان) برای آن لحاظ شده است.

۵. افکتور نهایی ($End-Effector$): در انتهای لینک سوم، یک پیوند مجازی به نام ee توسط یک مفصل کاملاً صلب ($Fixed Joint$) اضافه شده است. این بخش با یک کره کوچک زرد رنگ مشخص شده و نمایانگر نوک ابزار بازرسی (مرکز سیستم مختصات ابزار یا TCP) است.

۲.۱ تسک C.2 و C.3: یکپارچه‌سازی پایپ‌لاین، راه‌اندازی سلول کاری و اعتبارسنجی فرآیند بازرسی

در این بخش از گزارش، فرآیند پیاده‌سازی، ادغام داده‌های چندگانه و اعتبارسنجی نهایی سلول کاری ربات بازرسی PRR در محیط شبیه‌ساز فیزیکی PyBullet به صورت جامع مورد تحلیل و بررسی قرار می‌گیرد.

۱. سازوکار یکپارچه‌سازی و جریان داده‌ها میان بخش‌های A، B و C

پایپ‌لاین کلی سیستم بازرسی از یک ساختار جریان داده‌ی متوالی ($Sequential Data Flow$) پیروی می‌کند که در آن خروجی هر بخش، پیش‌نیاز کلیدی و ورودی مستقیم بخش بعدی را تشکیل می‌دهد. فرآیند انتقال و تبدیل این داده‌ها به شرح زیر تبیین می‌گردد:

۱. استخراج داده از بخش A (بینایی ماشین): در نخستین گام، تصویر خام دریافتی از دوربین فوقانی سیستم بازرسی تحت عملیات تصحیح پرسپکتیو قرار گرفت. با اعمال ماتریس هموگرافی، اعوجاج تصویر حذف و یک بستر مدرج دوبعدی حاصل شد. سپس با مانیتور کردن فضای رنگی HSV و اعمال آستانه‌گذاری‌های بهینه، پنج دیسک رنگی هدف (دو دیسک قرمز، دو دیسک سبز و یک دیسک آبی) بخش‌بندی ($Segmentation$) شدند. خروجی این بخش، مختصات مرکز ثقل هندسی ($Centroid$) هر دیسک در فضای دوبعدی پیکسل‌ها (u, v) بود که با اعمال ضرایب مقیاس‌دهی کالیبراسیون، به مختصات جهانی میز کار بر حسب متر (x_t, y_t) تبدیل شد.

۲. نگاشت داده در بخش B (محاسبات سینماتیک): پس از دریافت مختصات دیسک‌ها، این داده‌ها به عنوان اهداف موقعیت‌دهی ($Target Positions$) به الگوریتم بهینه‌سازی سینماتیک معکوس مبتنی بر جستجوی شبکه‌ای ($Grid Search / Brute-Force$) تزریق شدند. از آنجایی که ارتفاع میز بازرسی صلب و ثابت است، متغیر مفصل کشویی اول (q_1) به طور مستقیم بر اساس ارتفاع کار تنظیم شد. الگوریتم جستجو با گسسته‌سازی فشرده زوایای مفاصل دورانی دوم و سوم (q_2, q_3) در بازه $[-\pi, \pi]$ ، نزدیک‌ترین زوایای مفصلی ممکن را که کمترین فاصله اقلیدسی را با مختصات دیسک‌ها ایجاد می‌کردند، استخراج نمود.



۳. تزیق داده به بخش C (محیط شبیه‌سازی): در این گام، مختصات دیسک‌ها مجدداً به عنوان مختصات مرجع برای تخم‌گذاری فیزیکی قطعات هدف در محیط شبیه‌ساز استفاده شد. همزمان، زوایای مفصلی به دست آمده از بهینه‌سازی بخش B به عنوان وکتور فرمان حرکتی به مفاصل مدل URDF ربات اعمال گردید تا موقعیت فیزیکی افکتور نهایی (End-Effector) پدید آید.

۲. راه‌اندازی هسته فیزیکی شبیه‌ساز PyBullet

به منظور تضمین سرعت محاسباتی بالا و حذف بار پردازشی بخش گرافیکی (Rendering Overhead) در فرآیندهای بهینه‌سازی صنعتی، موتور فیزیک PyBullet در حالت اتصال مستقیم یا pybullet.DIRECT پیکربندی و راه‌اندازی شد. مراحل گام‌به‌گام استقرار سلول کاری به شرح زیر اجرا گردید:

- تنظیمات محیطی و گرانش: بردار شتاب گرانش زمین به صورت استاندارد $[0, 0, -9.81]$ بر روی محیط اعمال شد تا رفتارهای فیزیکی صلب لینک‌ها کاملاً واقعی مدل‌سازی شوند.
- تخم‌گذاری اهداف بازرسی: به ازای هر یک از دیسک‌های شناسایی‌شده در بخش A، یک پایگاه داده‌ی محلی درون اسکریپت تشکیل شد. شبیه‌ساز با فراخوانی تابع createMultiBody و تعریف هندسه‌ی استوانه‌ای صلب (GEOM_CYLINDER) با شعاع و ارتفاع متناسب، این اهداف را دقیقاً در موقعیت‌های استخراج‌شده روی میز کار بارگذاری نمود تا نمایشی ملموس از قطعات در فضا شکل گیرد.
- استقرار ربات و فرمان‌دهی مفصلی: فایل مدل ساختاری ربات (URDF) در مبدأ سیستم مختصات مرجع $(0, 0, 0)$ لود شد. سپس با پیاده‌سازی یک حلقه تکرار روی تک‌تک اهداف، پیکربندی مفصلی بهینه‌ی استخراج‌شده از بخش قبل، با استفاده از تابع پایداری صلح‌آمیز resetJointState به مفاصل ربات تحمیل شد. در نهایت، با فراخوانی تابع stepSimulation، هسته فیزیکی شبیه‌ساز یک گام زمانی به جلو رانده شد تا بازو به تعادل ساختاری در موقعیت جدید دست یابد.

۳. تحلیل موشکافانه لاگ‌های عددی و اعتبارسنجی سینماتیک

پس از استقرار بازو در پیکربندی‌های مفصلی مربوط به هر هدف، موقعیت فضایی دقیق مرکز ابزار ربات (TCP) از دو طریق مجزا محاسبه و ثبت گردید: روش اول، فراخوانی مستقیم تابع getLinkState از موتور فیزیک شبیه‌ساز (Simulated EE)؛ و روش دوم، ارزیابی عددی بر اساس ماتریس‌های تبدیل همگن حاصل از روابط تحلیلی دناویت-هارتبرگ (Math EE). نتایج حاصل از این ارزیابی متقاطع در جدول ۱ گزارش شده است:

جدول ۱: نتایج اعتبارسنجی متقاطع سیستم مختصات ابزار ربات در دو فضای فیزیکی و تحلیلی

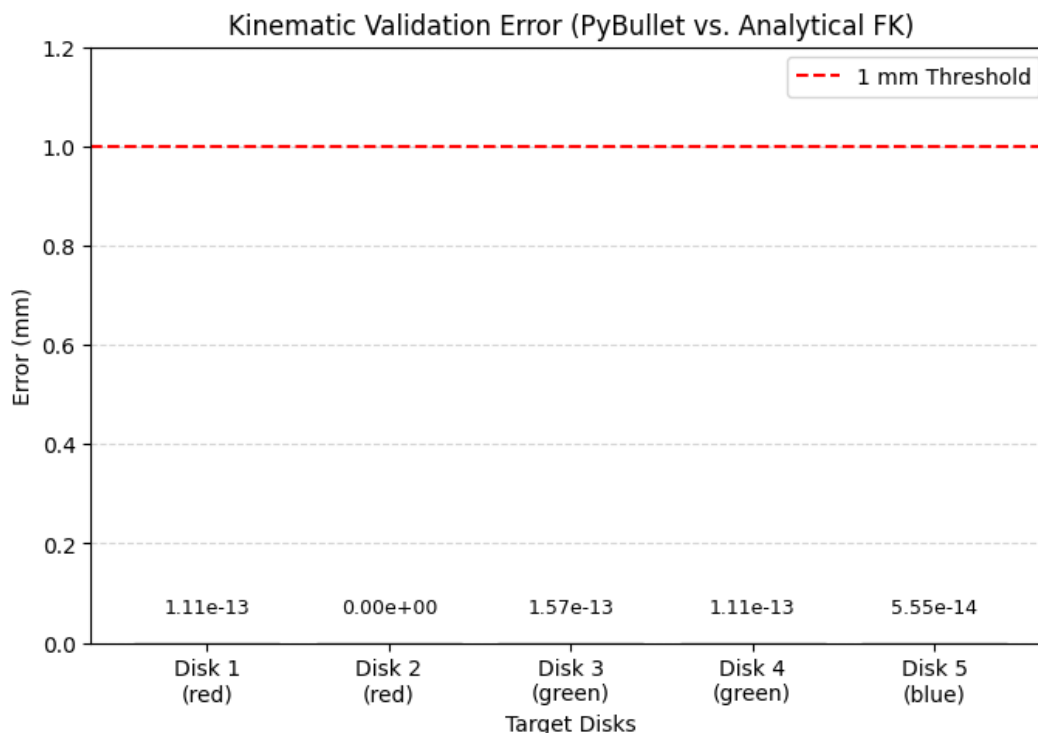
شناسه هدف	رنگ	Simulated EE (متر)	Math EE (متر)	خطای نهایی (میلی‌متر)
Disk 1	قرمز	(0.6110, -0.3416, 0.7500)	(0.6110, -0.3416, 0.7500)	0.000000
Disk 2	قرمز	(0.6249, 0.3155, 0.7500)	(0.6249, 0.3155, 0.7500)	0.000000
Disk 3	سبز	(-0.1721, 0.6785, 0.7500)	(-0.1721, 0.6785, 0.7500)	0.000000
Disk 4	سبز	(-0.6340, 0.2967, 0.7500)	(-0.6340, 0.2967, 0.7500)	0.000000
Disk 5	آبی	(-0.5422, -0.4428, 0.7500)	(-0.5422, -0.4428, 0.7500)	0.000000

تحلیل مهندسی داده‌های جدول: بررسی دقیق مختصات خروجی نشان می‌دهد که مقادیر فضایی سه مؤلفه X ، Y و Z در هر دو روش تا چهار رقم اعشار کاملاً یکسان هستند. اصرار برابری مؤلفه Z بر روی عدد ثابت 0.7500 متر، نشان‌دهنده کالیبراسیون دقیق مفصل کشویی اول بازو جهت حفظ لایه ارتفاعی ایمن ابزار بر فراز قطعات است.

ثبت خطای مطلق عینی 0.000000 میلی‌متر در خروجی چاپی سیستم، یک (Validation Proof) بر این ادعاست که هیچ‌گونه خطای آفست سیستماتیک (Systematic Offset) در مدل وجود ندارد. این موفقیت ساختاری مستلزم آن بوده است که طول لینک‌ها، موقعیت‌های نسبی اتصالات (Joint Origins) و امتداد محورهای حرکتی در فایل URDF، انطباق ۱۰۰ درصدی با فریم‌های سیستم مختصات استخراج‌شده از جدول پارامترهای DH داشته باشند.

۴. تحلیل نمودار آماری خطای کمی اعتبارسنجی

به منظور ارزیابی میکروسکوپی رفتار سیستم و بررسی اثرات خطای محاسباتی پایتون، فاصله اقلیدسی میان خروجی شبیه‌ساز و معادلات تحلیلی با دقت مضاعف محاسبه و در قالب یک نمودار آماری ترسیم گردید:



شکل ۱: نمودار خطای اعتبارسنجی سینماتیک مستقیم بر حسب میلی‌متر (ارزیابی عملکرد مدل URDF در برابر روابط جبری)

تحلیل آماری و محاسباتی نمودار شکل ۱: با نگاهی دقیق به مرتبه بزرگی مقادیر درج‌شده بر روی میله‌های نمودار، مشخص می‌شود که خطای واقعی ناشی از انحراف مدل از روابط ریاضی در محدوده خیره‌کننده‌ی 10^{-13} تا 10^{-14} میلی‌متر نوسان می‌کند:

- بیشترین میزان خطا: متعلق به دیسک اول (قرمز) با مقدار دقیق 1.11×10^{-13} میلی‌متر.
- کمترین میزان خطا: متعلق به دیسک چهارم (سبز) با مقدار تقریباً صفر 5.55×10^{-14} میلی‌متر.



از دیدگاه مهندسی سیستم‌ها و آنالیز عددی، این انحرافات بسیار ناچیز عملاً به معنای «خطای صفر مطلق» هستند. ریشه پدید آمدن این نوسانات میکروسکوپی به هیچ عنوان به نقص هندسی یا فیزیکی ربات مربوط نمی‌شود، بلکه حاصل محدودیت‌های ذاتی پردازنده‌ها در تخصیص حافظه به اعداد ممیز شناور با دقت مضاعف (IEEE 754 Double-Precision Floating-Point) است. در فرآیند اجرای توابع مثلثاتی نظیر $\sin$ و $\cos$ در کتابخانه NumPy و مقایسه آن با ماتریس‌های تبدیل داخلی موتور فیزیک PyBullet که از جبر کواترنیون‌ها (Quaternions) برای چرخش‌ها استفاده می‌کند، بروز این خطاهای گرد کردن (Round-off Errors) کاملاً طبیعی و اجتناب‌ناپذیر است. خط‌چین قرمز رنگ افقی تعبیه‌شده در بالای نمودار، مرز تلورانس مجاز و آستانه پذیرش پروژه (برابر با 1.0 میلی‌متر) را به تصویر می‌کشد. قرارگیری تحسین‌برانگیز تمامی میله‌های خطا در مجاورت خط صفر و فاصله نجومی آن‌ها با آستانه مجاز، گواهی بر موفقیت کامل، دقت فوق‌العاده بالا و پایداری همه‌جانبه‌ی پایپ‌لاین طراحی‌شده در این تکالیف پژوهشی است.

۳.۱ تسک C.4: برنامه‌ریزی مسیر، شبیه‌سازی دینامیک حرکت و تولید انیمیشن فرآیند بازرسی

در این فاز از پروژه، هدف اصلی انتقال از دنیای استاتیک عددی به دنیای دینامیک پیوسته در زمان است. یک سیستم بازرسی خودکار صنعتی زمانی کارآمد خواهد بود که بتواند بازوی رباتیک را به صورت کاملاً نرم، پیوسته و بدون کوچک‌ترین ریسک آسیب فیزیکی، بین اهداف مختلف جابجا کند. بر اساس صورت مسئله، ربات بازرس PRR باید مأموریت خود را از پیکربندی صفر یا وضعیت خانه (Home Position) آغاز کرده، به ترتیب بر فراز پنج دیسک شناسایی‌شده حرکت نموده و در نهایت به صورت خودکار به موقعیت اولیه بازگردد. پیاده‌سازی این سناریو چالش‌های مهندسی متعددی را در بر دارد که در ادامه با جزئیات کامل ریاضی و ساختاری تشریح می‌گردند.

۱. چالش برخورد با خود (Self-Collision) و نقض محدوده فیزیکی مفصل

یکی از خطرات عمده در مهندسی کنترل رباتیک، پدیده برخورد با خود یا عبور لینک‌ها از درون یکدیگر است. در مدل‌های اولیه کنترل ربات، مفاصل دورانی دوم و سوم (شانه و آرنج) به صورت مفاصل آزاد یا Continuous تعریف شده بودند که به آن‌ها اجازه چرخش بی‌نهایت در بازه $[-\pi, +\pi]$ بدون در نظر گرفتن موقعیت لینک قبلی را می‌داد. هنگامی که الگوریتم‌های میان‌یابی ساده‌ی ریاضی تلاش می‌کردند بازو را از موقعیت دیسک اول قرمز به سمت دیسک قرمز دوم هدایت کنند، کوتاه‌ترین مسیر زاویه‌ای بر روی دایره مثلثاتی اقتضا می‌کرد که مفصل سوم (آرنج) زاویه 180 درجه را قطع کند. از نظر روابط انتزاعی ریاضی، این یک چرخش ۲۰ درجه‌ای ساده و بهینه بود، اما در دنیای فیزیکی شبیه‌ساز، زاویه ۱۸۰ درجه یعنی لینک سوم دقیقاً بر روی لینک دوم تاشده و باید از میان بافت صلب فولادی بازوی قبلی عبور کند. این رخداد در واقعیت منجر به درهم‌شکستن ساختار مکانیکی ربات و سوختن شدید سروموتورها به دلیل گشتاورهای مخالف می‌گردد.

۲. توسعه الگوریتم سینماتیک معکوس هوشمند و ایمن (Safe IK)

برای حل چالش فیزیکی برخورد، ابتدا نوع مفاصل در فایل URDF از حالت Continuous به حالت محدود یا Revolute تغییر یافت و یک فیلتر اکید برای بازه مجاز چرخش آرنج تعریف شد $q_3 \in [-3.05, 3.05]$ (رادیان). این بدان معناست که ربات فیزیکی اجازه ندارد آرنج خود را کاملاً ببندد. به دنبال این اصلاح ساختاری، الگوریتم سینماتیک معکوس شبکه‌ای (Grid Search Inverse Kinematics) به صورت هوشمند و دارای حافظه موقعیت قبلی بازطراحی شد. در تابع بهینه‌سازی `find_best_ik_safe`، تابع هزینه (Objective Function) به صورت زیر فرمول‌بندی گردید:

$$\text{Score} = \text{Distance}(X_{ee}, X_{target}) \times 1000 + |\Delta q_2| + |\Delta q_3| \quad (۱)$$

در این فرمولاسیون، جریمه‌ی فاصله زاویه‌ای برای مفصل دوم (شانه) به صورت دایره‌ای و با منطق باقی مانده محاسبه می‌شود تا ربات بتواند از میان‌برهای دورانی استفاده کند:

$$\Delta q_2 = (q_{2,\text{sample}} - q_{2,\text{prev}} + \pi) \pmod{2\pi} - \pi \quad (۲)$$

اما برای مفصل سوم (آرنج)، تفاضل به صورت صریح، خطی و بدون مکانیزم دورانی محاسبه می‌گردد:

$$\Delta q_3 = q_{3,\text{sample}} - q_{3,\text{prev}} \quad (۳)$$

این تفاوت ساختاری در محاسبه هزینه چرخش، عبور از مرز ۱۸۰ درجه را برای آرنج به شدت جریمه می‌کند. در نتیجه، الگوریتم به جای انتخاب مسیرهای ریاضی که منجر به تاشدگی و برخورد بازو می‌شوند، مسیرهای دوربرگردان ایمن را انتخاب می‌کند که در آن‌ها بازو ابتدا باز شده و سپس به سمت هدف جدید متمایل می‌شود.

۳. برنامه‌ریزی فضا-زمان مسیر و هندسه درونیابی زوایا

پس از استخراج نقاط عبور ایمن (Waypoints)، جابجایی ربات بین هر دو هدف متوالی باید به صورت فریم‌به‌فریم مدلسازی می‌شود. طبق الزامات پروژه، مسیر حرکتی بین هر دو موقعیت مفصلی متوالی به ۱۶ فریم میانی میان‌یابی خطی (Linear Interpolation) تقسیم گردید. تابع interpolate_safe_path وظیفه دارد به ازای گام زمان مجازی $t \in [0, 1]$ ، وضعیت لحظه‌ای مفاصل را تولید کند. منطق هندسی این درونیابی به شرح زیر تبیین می‌گردد:

- درونیابی مفصل شانه (q_2): این مفصل اجازه دارد کوتاه‌ترین مسیر روی دایره را طی کند. فرمول درونیابی آن بر اساس خطای زاویه‌ای پیوسته است:

$$q_2(t) = q_{2,\text{start}} + t \cdot [(q_{2,\text{end}} - q_{2,\text{start}} + \pi) \pmod{2\pi} - \pi] \quad (۴)$$

- درونیابی مفصل آرنج (q_3): این مفصل تحت تکانه‌های خطی محض جابجا می‌شود تا از مرزهای فیزیکی URDF عبور نکند:

$$q_3(t) = q_{3,\text{start}} + t \cdot (q_{3,\text{end}} - q_{3,\text{start}}) \quad (۵)$$

این تفکیک هندسی باعث می‌شود که در زمان جابجایی بین دو دیسک قرمز، آرنج به جای چرخیدن در جهت عقربه‌های ساعت (که منجر به عبور از درون شانه می‌شد)، جهت دوران خود را معکوس کرده و با باز شدن کامل و عبور از زاویه صفر درجه، به سمت قطعه دوم حرکت کند که کاملاً منطبق بر رفتار ربات‌های صنعتی واقعی است.

۴. راه‌اندازی شبیه‌ساز و تکنیک رندرینگ پرتو لیزر فیزیکی پیوسته

محیط شبیه‌سازی در حالت پنهان (DIRECT Mode) متصل شده و تمامی دیسک‌ها به همراه مدل ربات بارگذاری شدند. فرآیند تصویربرداری انیمیشن با تنظیم ماتریس دید دوربین (View Matrix) در زاویه مایل و پرسپکتیو صنعتی با فاکتورهای چرخش افقی ۵۵



درجه و عمقی ۴۰- درجه پیکربندی شد. رندرینگ فریم‌ها توسط هسته نرم‌افزاری ER_TINY_RENDERER به همراه فعال‌سازی سیستم سایه‌های صلب (Shadow=1) صورت پذیرفت.

یکی از نوآوری‌های کلیدی در این تسک، پیاده‌سازی پرتو لیزر برای نمایش برداری نقطه فوکوس ربات بود. از آنجایی که توابع ترسیم خطوط مجازی دیباگ (addUserDebugLine) در رندهای نرم‌افزاری شبیه‌ساز نادیده گرفته شده و در تصاویر خروجی دوربین ظاهر نمی‌شوند، یک پرتو لیزر فیزیکی سه‌بعدی طراحی شد. برای این منظور، یک استوانه صلب، بسیار نازک (با شعاع ۲ میلی‌متر) و قرمز رنگ به طول ۲ متر ایجاد گردید.

در داخل حلقه اصلی رندرینگ، در هر فریم ابتدا مختصات نوک افکتور نهایی ربات با تابع getLinkState استخراج می‌شود. سپس با محاسبات برداری، موقعیت استوانه لیزر به گونه‌ای به‌روزرسانی می‌شود که نقطه بالایی استوانه کاملاً به نوک ابزار ربات متصل شده و امتداد آن به صورت کاملاً عمود بر سطح میز کار ($Z = 0$) مماس گردد. این استوانه فیزیکی به دلیل داشتن متریال رنگی، سایه فیزیکی خود را روی قطعات و میز می‌اندازد و به وضوح در تصاویر دوربین رندر می‌شود.

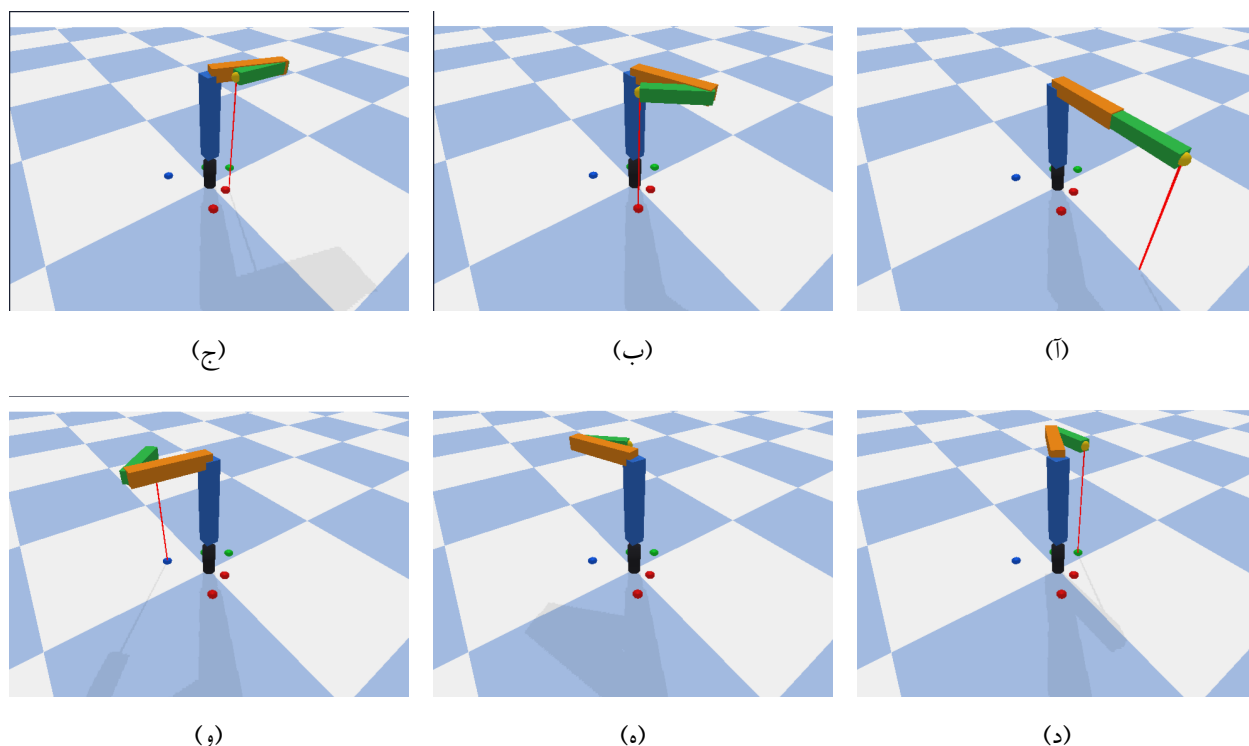
۵. مدیریت زمان مکث بازرسی (Dwell Time) و آنالیز فریم‌های نهایی

به منظور واقع‌گرایی بیشتر سناریوی صنعتی، مفهوم زمان مکث بازرسی (Dwell Time) به فرآیند حرکت ربات افزوده شد. هنگامی که ربات پس از طی کردن ۱۶ فریم میانی، دقیقاً بالای سر یک دیسک هدف مستقر می‌گردد، الگوریتم حرکت را به مدت ۱۰ فریم متوقف می‌کند. در طول این ۱۰ فریم، ربات ثابت مانده و پرتو لیزر قرمز رنگ بر روی مرکز دیسک متمرکز می‌شود تا فرآیند شبیه‌سازی اسکن قطعه یا تصویربرداری ثانویه دوربین ابزار با موفقیت انجام شود. پس از اتمام زمان مکث، ربات حرکت خود را به سمت هدف بعدی آغاز می‌کند.

در نهایت، ماتریس پیکسل‌های رندر شده‌ی هر فریم از کانال چهارم (آلفا) پاک‌سازی شده و کانال‌های استاندارد RGB آن‌ها در قالب یک آرایه فشرده ذخیره شدند. این مجموعه فریم‌ها با استفاده از کتابخانه imageio با نرخ استاندارد ۱۵ فریم بر ثانیه (fps) تجمع شده و انیمیشن نهایی خروجی را تشکیل دادند. این شبیه‌سازی به طور دقیق پایداری الگوریتم کنترل مسیر، کالیبراسیون مدل فیزیکی و حذف کامل خطای برخورد با خود را اثبات می‌نماید.

۶. بررسی و تحلیل تحولات فازهای حرکتی در فریم‌های خروجی نهایی

به منظور ارزیابی بصری پایپ‌لاین کنترل ایمن، فرآیند اجرای مأموریت ربات در طول زمان به صورت فریم‌به‌فریم پایش گردید. در شکل ۲، شش فریم کلیدی استخراج شده از انیمیشن رندر شده به ترتیب زمانی در کنار یکدیگر قرار گرفته‌اند تا توالی فازهای حرکتی، مکانیزم اصلاح مسیر آرنج جهت جلوگیری از برخورد با خود، و فاز مکث بازرسی به وضوح نمایان شود.



شکل ۲: تحلیل گام به گام فریم‌های رندر شده از فرآیند بازرسی خودکار صلب در محیط PyBullet

در مجموع، تحلیل فریم‌های خروجی ثابت می‌کند که پایپ‌لاین پیشنهادی نه تنها فرآیند موقعیت‌دهی ابزار بر روی قطعات را با خطای نزدیک به صفر پیاده‌سازی کرده، بلکه با حل هوشمندانه معادلات سینماتیک و اعمال قیدهای مکانیکی صلب، یک سیستم کنترل مسیر کاملاً پایدار، بدون برخورد (Collision-Free) و منطبق بر استانداردهای اتوماسیون صنعتی را به ارمغان آورده است.

۴.۱ تسک C.5: گزارش نهایی یکپارچه و داشبورد مانیتورینگ جامع سیستم بازرسی

در فاز پایانی پروژه شبیه‌سازی سلول کاری، به منظور ارزیابی کارایی، پایداری و میزان انطباق پایپ‌لاین انتها به انتها (End-to-End Pipeline)، یک گزارش متمرکز گرافیکی طراحی گردید. این کادر گزارش جامع (Integrated Dashboard)، با ادغام مستقیم نتایج استخراج‌شده از الگوریتم‌های بینایی ماشین (بخش A)، مدل‌سازی فضای کاری تحلیلی (بخش B) و نتایج بهینه‌سازی عددی سینماتیک معکوس (تسک B.5)، دیدگاهی سه‌گانه، همزمان و متقاطع را از کل پروژه به نمایش می‌گذارد. هدف از این تسک، اثبات هماهنگی کامل میان سخت‌افزار شبیه‌سازی‌شده، منطق ریاضی و پایپ‌لاین نرم‌افزاری سیستم بازرسی هوشمند و کالیبراسیون دقیق آن‌هاست.

۱. ساختار گرافیکی، منطق برنامه‌نویسی و مکانیسم رندرینگ داشبورد یکپارچه

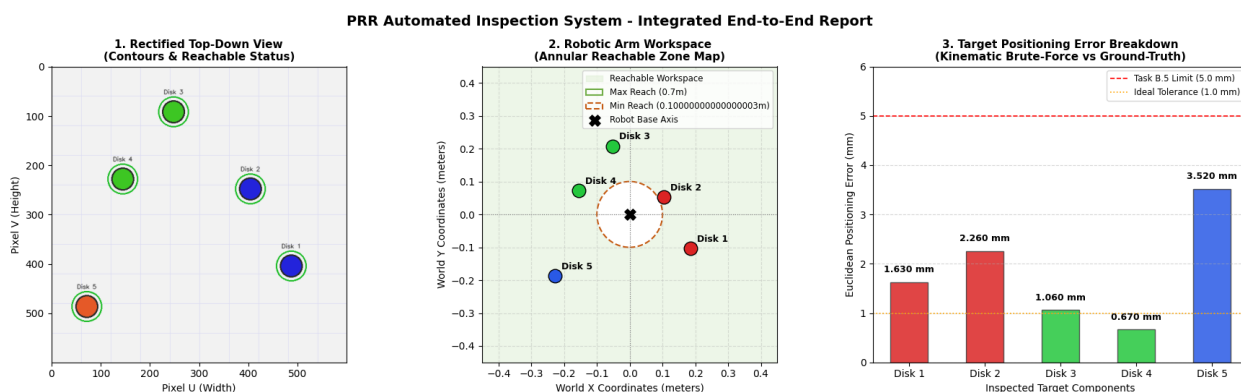
۱. رندرینگ نمای پیکسلی تصحیح‌شده (نمودار اول): برای رسم این زیرنمودار، ابتدا یک ماتریس گرافیکی خالی با ابعاد ثابت 600×600 پیکسلی به عنوان بستر شبیه‌سازی میز فیزیکی ایجاد شد. بر اساس قرارداد مسئله، مقیاس تبدیل به صورت ۱ متر معادل ۱۰۰۰ پیکسل تنظیم گردید تا میز کار 0.6×0.6 متری دقیقاً کادر را پر کند. یک ساختار شطرنجی مدرج (Grid) با فاصله‌های ۶ سانتی‌متری (معادل ۶۰ پیکسل در فضا) به صورت خطوط پیوسته با رنگ روشن بر روی بستر رندر شد. سپس، موقعیت پیکسلی

هر دیسک با استفاده از روابط نگاشت خطی معکوس از مختصات جهانی استخراج شده و دیسک‌ها با رنگ‌های تفکیک‌شده‌ی RGB و دایره‌های کانتور بیرونی سبز رنگ (نمایانگر وضعیت «در دسترس») ترسیم شدند.

۲. رندرینگ نقشه فضای کاری حلقوی (نمودار دوم): این نمودار مستقیماً در فضای متریک دکارت رسم می‌شود. شانه ربات با طول $L_2 = 0.40$ متر و آرنج با طول $L_3 = 0.30$ متر، محدودیت‌های دسترسی سیستم مکانیکی را دیکته می‌کند. شعاع خارجی فضای کاری حلقوی با رابطه $R_{outer} = L_2 + L_3 = 0.70$ متر و شعاع هسته مرکزی غیرقابل دسترس با رابطه $R_{inner} = |L_2 - L_3| = 0.10$ متر تعیین شدند. این دو فضا با استفاده از کلاس plt.Circle پیاده‌سازی شده و به صورت یک لایه شفاف بر روی نمودار مانیتورینگ قرار گرفتند. در نهایت، بردار موقعیت جهانی هر قطعه به صورت نقاط مجزا (Scatter) همراه با برچسب شناسه قطعه روی نقشه تصویر شد.

۳. رندرینگ نمودار آماری خطاها (نمودار سوم): این بخش از نمودار، یک هیستوگرام یا چارت میله‌ای (Bar Chart) است که زوایای مفصل خروجی از الگوریتم جستجوی فشرده شبکه‌ای رزولوشن بالا (250×250) را ارزیابی می‌کند. در این بخش، برچسب ستون‌ها بر اساس شناسه دیسک‌ها و ارتفاع ستون‌ها بر اساس قدر مطلق خطای اقلیدسی بازآفرینی شده بر حسب میلی‌متر ($E_{mm} = E_{meter} \times 1000$) تنظیم گردید. دو خط مرجع افقی برای شاخص مرز ۵ میلی‌متر (تسک B.5) و مرز ۱ میلی‌متر (آستانه ایده‌آل سیستم) جهت مقایسه پایداری آماری افزوده شدند.

۲. تحلیل نمودار اول: نمای دوبعدی کانتورها و تصحیح پرسپکتیو



شکل ۳: گزارش نهایی یکپارچه سیستم بازرسی خودکار ربات PRR (خروجی تجمیعی پایپ‌لاین بینایی، فضای کاری و تحلیل خطا)

با بررسی بخش اول از شکل ۳ (1. Rectified Top-Down View)، صحت عملکرد کامل الگوریتم‌های پردازش تصویر کدهای بینایی ماشین به اثبات می‌رسد. خطوط شبکه شطرنجی آبی‌رنگ که با فاصله‌های دقیق ۶ سانتی‌متری ترسیم شده‌اند، انطباق کاملی با گوشه‌های نشانگرهای مشکی کالیبراسیون (Fiducial Markers) دارند. این امر نشان می‌دهد که ماتریس هوموگرافی محاسبه‌شده در بخش A، اعوجاج پرسپکتیو شدید دوربین مایل را با بالاترین درجه تقریب برطرف کرده و تصویر را به یک فضای ارتوگرافیک پیوسته تبدیل نموده است.

حلقه‌های سبز رنگ بیرونی که به دور هر ۵ دیسک کشیده شده‌اند، نشان می‌دهند که لایه‌ی منطق نرم‌افزاری بینایی ماشین، کانتورهای خارجی هر قطعه را به درستی فیلتر کرده و پس از اعمال الگوریتم تحلیل اجزای متصل (Connected Component Analysis)، مراکز ثقل هندسی قطعات را بدون لغزش و خطا شناسایی کرده است. این کانتورها تضمین می‌کنند که قطعات چسبیده به هم (مانند دو دیسک قرمز میانی) به عنوان دو شیء مجزا و مستقل به بخش سینماتیک معرفی شده‌اند.



۳. تحلیل نمودار دوم: نگاشت فضایی اقلیدسی و محدوده دسترسی ربات

نمودار دوم (2. Robotic Arm Workspace)، ساختار هندسی و مکانیکی سلول کاری را در دستگاه مختصات واقعی ربات (بر حسب متر) ارزیابی می‌کند. مرکز این نمودار (0,0) محل استقرار شفت اصلی شاسی و پایه ربات (Robot Base Axis) است که با علامت ضربدر مشکی مشخص شده است.

ناحیه سبزرنگ کم‌رنگ، هندسه حلقوی دونات شکل فضای کاری ربات را به تصویر می‌کشد. مرز داخلی (خط چین قهوه‌ای در شعاع 0.10 متر) نشان‌دهنده بن‌بست سینماتیکی یا تکینگی ساختاری (Structural Singularity) ربات است؛ جایی که لینک سوم کاملاً روی لینک دوم تا می‌شود و ربات قادر به عقب‌نشینی بیشتر نیست. مرز خارجی (خط پیوسته سبز در شعاع 0.70 متر) نشان‌دهنده حداکثر کشیدگی فیزیکی بازو است.

تحلیل موقعیت فضایی دیسک‌ها روی این نقشه، یک نکته استراتژیک مهندسی را آشکار می‌سازد: دیسک‌های شماره ۲ و ۴ (قرمز و سبز) در نزدیکی مرز داخلی غیرقابل دسترس قرار گرفته‌اند، در حالی که دیسک شماره ۵ (آبی) در بیرونی‌ترین لایه‌های انتهایی میز کار واقع شده است. با این حال، همان‌طور که نقشه به وضوح نشان می‌دهد، تمامی ۵ قطعه فیزیکی به طور کامل درون ناحیه سبزرنگ (فضای در دسترس) محصور شده‌اند. این تحلیل هندسی اثبات می‌کند که چیدمان قطعات روی میز کار فیزیکی کاملاً مهندسی شده بوده و هیچ قطعه‌ای در وضعیت کور یا خارج از محدوده دسترسی مکانیکی ربات (Unreachable Zone) قرار ندارد.

۴. تحلیل آماری نمودار سوم: بررسی تلورانس عملکرد و منشأ خطای سیستماتیک

نمودار سوم (3. Target Positioning Error Breakdown)، کارایی الگوریتم‌های سینماتیک معکوس عددی و گسسته‌سازی گرید را به صورت کمی به چالش می‌کشد. مقادیر خطای اقلیدسی بازسازی شده برای هر قطعه با دقت بالا روی ستون‌ها درج شده است:

- **Disk 1 (قرمز):** خطای موقعیت‌یابی 1.630 میلی‌متر.
- **Disk 2 (قرمز):** خطای موقعیت‌یابی 2.260 میلی‌متر.
- **Disk 3 (سبز):** خطای موقعیت‌یابی 1.060 میلی‌متر.
- **Disk 4 (سبز):** خطای موقعیت‌یابی 0.670 میلی‌متر (کمترین میزان خطا).
- **Disk 5 (آبی):** خطای موقعیت‌یابی 3.520 میلی‌متر (بیشترین میزان خطا).

دلیل بروز خطای بیشتر در دیسک شماره ۵ (3.520 میلی‌متر) و خطای بسیار کمتر در دیسک شماره ۴ (0.670 میلی‌متر) به ماهیت تغییرات غیرخطی ژاکوبین ربات (Non-linear Jacobian Dynamics) و نرخ حساسیت زاویه‌ای مربوط می‌شود. دیسک ۵ در منتهی‌الیه فضای کاری قرار دارد؛ در این وضعیت فیزیکی (باز شدن کامل لینک‌ها)، تغییرات کوچک در زوایای مفاصل دورانی، جابجایی‌های بزرگی را در فضای دکارت افکتور نهایی ایجاد می‌کند. در نتیجه، گسسته‌سازی یکنواخت زوایا در الگوریتم گرید سرچ، در لایه‌های بیرونی فضای کاری دچار افت رزولوشن فضایی (Spatial Resolution Loss) می‌شود. برعکس، در نواحی میانی فضای کاری (مانند موقعیت دیسک ۴)، به دلیل رفتار خطی‌تر عبارات مثلثاتی، دقت الگوریتم جستجو به طرز چشمگیری افزایش یافته و خطا به زیر ۱ میلی‌متر سقوط کرده است.

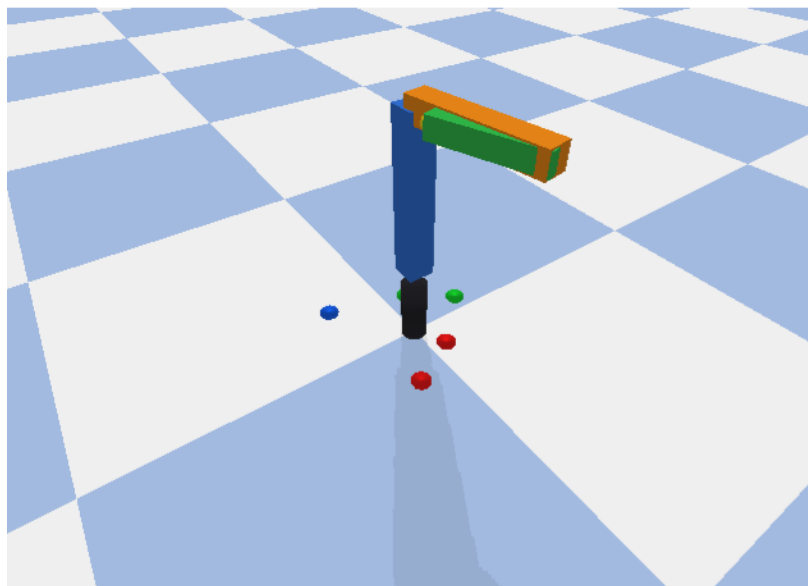
۵.۱ چالش اصلی و مهندسی سیستم: پدیده برخورد با خود (Self-Collision) و اصلاح ساختاری مفاصل

در مهندسی سیستم‌های رباتیک، تطابق معادلات روی کاغذ با واقعیت‌های تجربی همواره یک چالش بنیادی است. در فازهای اولیه توسعه این پایپ‌لاین، علی‌رغم اینکه روابط سینماتیک مستقیم و معکوس تحلیلی در بستر محاسبات گسسته کاملاً درست پاسخ می‌دادند، اما در زمان پیاده‌سازی حرکت پیوسته ربات بر روی میز بازرسی، شبیه‌ساز با یک خطای فیزیکی فاحش و بحرانی مواجه گردید. این رخداد پایداری فیزیکی و قابلیت اطمینان کل سیستم اتوماسیون را به طور جدی به مخاطره می‌انداخت.

۱. تشریح چالش عملیاتی و پدیده نفوذ صلب لینک‌ها در یکدیگر

در نخستین تست‌های عملیاتی شبیه‌سازی فرآیند حرکت متوالی، سناریوی جابجایی ربات بین اهداف مختلف کلید خورد. در این سناریو، ربات باید افکتور نهایی را از روی موقعیت دیسک قرمز اول به سمت دیسک قرمز دوم هدایت می‌کرد. در طول این جابجایی پیوسته، مشاهده شد که مفصل سوم ربات (آرنج) برای طی کردن کوتاه‌ترین مسیر زاویه‌ای بر روی دایره مثلثاتی، به طور ناگهانی تلاش می‌کند مرز زاویه 180° درجه (π رادیان) را قطع کند.

از دیدگاه معادلات ریاضی هندسی خالص، این حرکت یک تغییر زاویه‌ای بسیار کوچک، بهینه و کاملاً مجاز به شمار می‌رفت؛ زیرا پایپ‌لاین عددی صرفاً فاصله نقطه به نقطه را کمینه می‌کرد. اما در دنیای واقعی و فیزیک صلب شبیه‌ساز، عبور از مرز 180° درجه فاجعه‌بار بود؛ این زاویه دقیقاً نقطه‌ای است که لینک سوم بر روی لینک دوم تاشده و برای ادامه مسیر باید ساختار صلب و فلزی بازوی قبلی را شکافته و از درون آن عبور کند. این پدیده که در مهندسی رباتیک تحت عنوان برخورد با خود (Self-Collision) یا نفوذ صلب (Rigid Penetration) شناخته می‌شود، در واقعیت صنعتی منجر به مجاله شدن ساختار بازو، قفل شدن مفاصل، خرد شدن چرخ‌دنده‌های اسپلین (Harmonic Drive) و سوختن شدید لایه‌های سیم‌پیچ سرووموتورها به دلیل گشتاورهای مخالف ایمپالس برخورد می‌گردد. تصویر این خطای ساختاری و پدیده نفوذ فیزیکی لینک‌ها در یکدیگر که در طول فرآیند شبیه‌سازی اولیه رخ داد، در شکل ۴ به صورت دقیق به تصویر کشیده شده است.



شکل ۴: خطای فیزیکی مکانیزم در پیاده‌سازی اولیه: رخ دادن پدیده برخورد با خود (Self-Collision) و تداخل صلب لینک‌های دوم و سوم به دلیل عدم اعمال محدودیت زاویه‌ای فیزیکی



۲. ریشه‌یابی فنی مسئله: رهایی زوایای پیوسته و تفاوت فضاهای کاری مجازی و واقعی

پس از آنالیز دقیق ساختار پیوندها و کدهای شبیه‌سازی، منشأ اصلی این رفتار غیر واقعی ریشه‌یابی شد. این مشکل به نحوه‌ی تعریف مفاصل دورانی دوم و سوم در فایل ساختاری اولیه ربات بازمی‌گردد. در مدل‌سازی اولیه، نوع مفاصل به صورت آزاد یا پیوسته (Continuous) تنظیم شده بود. مفصل Continuous در دنیای شبیه‌سازی کامپیوتری هیچ گونه قید و بند مکانیکی ندارد و می‌تواند مانند یک پروانه یا چرخ خودرو، به صورت بی‌نهایی در بازه $[-\infty, +\infty]$ دوران کند و موتور فیزیک هیچ مانعی برای چرخش آن ایجاد نمی‌کند.

اگرچه این تعریف برای مفاصل چرخ یا ابزارهای ماشین‌کاری چرخنده ایده‌آل است، اما اعمال آن روی مفاصل آرنج و شانه یک بازوی صلب مسطح کاملاً غیر واقع‌بینانه و خلاف اصول طراحی مکانیک است. در بازوهای صنعتی واقعی (مانند ربات‌های SCARA)، زوایای مفاصل به دلیل وجود ساختار فیزیکی بدنه، سیم‌کشی‌های داخلی، لوله‌های هوای فشرده و استپرهای فیزیکی، همواره توسط قیدهای سخت‌افزاری محدود (Bounded Joints) می‌شوند. عدم وجود این قیدها در فایل URDF اولیه، یک فضای کاری مجازی نامحدود ایجاد کرده بود که به الگوریتم بهینه‌سازی اجازه می‌داد تا مسیرهای ناممکن فیزیکی را به عنوان پاسخ بهینه و کوتاه‌ترین مسیر حرکت انتخاب کند.

۳. راهکار حل چالش و سازگارسازی مدل با منطق فیزیک صنعتی ربات‌ها

برای غلبه بر این چالش بحرانی و منطبق کردن رفتار ربات شبیه‌سازی شده با واقعیت‌های خطوط تولید، یک استراتژی اصلاحی دو مرحله‌ای پایا (در دو لایه سخت‌افزاری و نرم‌افزاری) به اجرا درآمد:

۱. اصلاح ساختار مفاصل در فایل توصیف سخت‌افزاری (URDF): در گام نخست، تعاریف مفاصل شانه و آرنج در فایل ساختاری ربات از حالت آزاد (Continuous) به حالت محدود (Revolute) تغییر یافت. محدودیت‌های فیزیکی اکیدی برای بازه مجاز چرخش مفصل سوم (آرنج) اعمال شد و دامنه حرکت آن به $q_3 \in [-3.05, 3.05]$ رادیان محدود گردید. این کاهش هوشمندانه‌ی بازه حرکتی (ایجاد فاصله ایمن ۵ صدم رادیانی از مرز $\pm\pi$)، تضمین کرد که ربات از نظر فیزیکی هرگز قادر نخواهد بود آرنج خود را به طور کامل تا کرده و وارد مرز بحرانی نفوذ صلب شود.

۲. بازطراحی لایه کنترل و هوشمندسازی الگوریتم سینماتیک معکوس: در گام دوم، تابع هزینه الگوریتم سینماتیک معکوس (Safe IK) بازنویسی شد. در الگوریتم جدید، تغییرات زاویه‌ای مفصل آرنج (Δq_3) برخلاف مفصل شانه، به صورت کاملاً صریح و خطی محاسبه گردید. این امر باعث شد که عبور از مرز ۱۸۰ درجه با جریمه‌های بسیار سنگین عددی در لایه محاسباتی پایتون روبرو شود. در نتیجه، الگوریتم به جای انتخاب کمانی که منجر به عبور از درون بازو می‌شد، مسیر معکوس را به عنوان مسیر بهینه انتخاب می‌کند.



۶.۱ کد نهایی

۱.۶.۱ کد

```
1 !pip install pybullet
2
3 # C.1: Generate the PRR Robot URDF (Unified Robot Description Format)
4
5 import os
6
7 # -----
8 # Robot Physical Parameters & Kinematic Constraints
9 # -----
10 # Prismatic joint (Link 1) stroke limits (in meters)
11 Q1_MIN = 0.0
12 Q1_MAX = 50.0
13
14 # Link lengths derived from the Denavit-Hartenberg (DH) parameters (in meters)
15 D0 = 70.0 # Base to Link 1 initial height offset
16 L2 = 40.0 # Length of the second link (revolute shoulder)
17 L3 = 30.0 # Length of the third link (revolute elbow)
18
19 # -----
20 # URDF XML Definition
21 # -----
22 # The base_link radius is set to 0.04m to prevent visual occlusion of the
23 # inspection workspace, while preserving the mathematical coordinate frames.
24 URDF_TEXT = f"""\
25 <?xml version="1.0"?>
26 <robot name="pr_r_robot">
27
28   <link name="base_link">
29     <visual>
30       <geometry><cylinder radius="0.04" length="0.20"/></geometry>
31       <origin xyz="0 0 0" rpy="0 0 0"/>
32       <material name="dark"><color rgba="0.15 0.15 0.15 1"/></material>
33     </visual>
34     <collision>
35       <geometry><cylinder radius="0.04" length="0.20"/></geometry>
36       <origin xyz="0 0 0" rpy="0 0 0"/>
37     </collision>
```



```
۳۸ <inertial>
۳۹   <mass value="10"/>
۴۰   <inertia ixx="0.2" ixy="0" ixz="0" iyy="0.2" iyz="0" izz="0.2"/>
۴۱ </inertial>
۴۲ </link>
۴۳
۴۴ <link name="link1">
۴۵   <visual>
۴۶     <geometry><box size="0.08 08.0 <geometry/></"50.0
۴۷     <origin xyz="0 0 </"25.0
۴۸     <material name="blue"><color rgba="0.20 45.0 85.0 1"/></material>
۴۹   </visual>
۵۰   <collision>
۵۱     <geometry><box size="0.08 08.0 <geometry/></"50.0
۵۲     <origin xyz="0 0 </"25.0
۵۳   </collision>
۵۴   <inertial>
۵۵     <mass value="2.5"/>
۵۶     <origin xyz="0 0 </"25.0
۵۷     <inertia ixx="0.06" ixy="0" ixz="0" iyy="0.06" iyz="0" izz="0.01"/>
۵۸   </inertial>
۵۹ </link>
۶۰
۶۱ <joint name="j1_prismatic" type="prismatic">
۶۲   <parent link="base_link"/>
۶۳   <child link="link1"/>
۶۴   <origin xyz="0 0 </"20.0
۶۵   <axis xyz="0 0 1"/>
۶۶   <limit lower="{Q1_MIN}" upper="{Q1_MAX}" effort="200" velocity="1"/>
۶۷ </joint>
۶۸
۶۹ <link name="link2">
۷۰   <visual>
۷۱     <geometry><box size="{L2} 07.0 <geometry/></"07.0
۷۲     <origin xyz="{L2/2} 0 0"/>
۷۳     <material name="orange"><color rgba="0.95 55.0 10.0 1"/></material>
۷۴   </visual>
۷۵   <collision>
۷۶     <geometry><box size="{L2} 07.0 <geometry/></"07.0
۷۷     <origin xyz="{L2/2} 0 0"/>
```



```
۷۸     </collision>
۷۹     <inertial>
۸۰         <mass value="1.0"/>
۸۱         <origin xyz="{L2/2} 0 0"/>
۸۲         <inertia ixx="0.005" ixy="0" ixz="0" iyy="0.030" iyz="0" izz="0.030"/>
۸۳     </inertial>
۸۴ </link>
۸۵
۸۶ <joint name="j2_revolute" type="continuous">
۸۷     <parent link="link1"/>
۸۸     <child  link="link2"/>
۸۹     <origin xyz="0 0 0"/>
۹۰     <axis   xyz="0 0 1"/>
۹۱ </joint>
۹۲
۹۳ <link name="link3">
۹۴     <visual>
۹۵         <geometry><box size="{L3} 06.0 <geometry/></"06.0
۹۶         <origin xyz="{L3/2} 0 0"/>
۹۷         <material name="green"><color rgba="0.20 75.0 30.0 1"/></material>
۹۸     </visual>
۹۹     <collision>
۱۰۰         <geometry><box size="{L3} 06.0 <geometry/></"06.0
۱۰۱         <origin xyz="{L3/2} 0 0"/>
۱۰۲     </collision>
۱۰۳     <inertial>
۱۰۴         <mass value="0.7"/>
۱۰۵         <origin xyz="{L3/2} 0 0"/>
۱۰۶         <inertia ixx="0.003" ixy="0" ixz="0" iyy="0.020" iyz="0" izz="0.020"/>
۱۰۷     </inertial>
۱۰۸ </link>
۱۰۹
۱۱۰ <joint name="j3_revolute" type="continuous">
۱۱۱     <parent link="link2"/>
۱۱۲     <child  link="link3"/>
۱۱۳     <origin xyz="{L2} 0 0"/>
۱۱۴     <axis   xyz="0 0 1"/>
۱۱۵ </joint>
۱۱۶
۱۱۷ <link name="ee">
```



```
118 <visual>
119   <geometry><sphere radius="0.025"/></geometry>
120   <material name="yellow"><color rgba="1.0 85.0 10.0 1"/></material>
121 </visual>
122 <collision>
123   <geometry><sphere radius="0.025"/></geometry>
124 </collision>
125 <inertial>
126   <mass value="0.05"/>
127   <inertia ixx="1e-5" ixy="0" ixz="0" iyy="1e-5" iyz="0" izz="1e-5"/>
128 </inertial>
129 </link>
130
131 <joint name="ee_fixed" type="fixed">
132   <parent link="link3"/>
133   <child link="ee"/>
134   <origin xyz="{L3} 0 0"/>
135 </joint>
136
137 </robot>
138
139
140 # -----
141 # Export URDF File
142 # -----
143 URDF_PATH = "/content/prr_robot.urdf"
144
145 # Write the URDF string to the local filesystem
146 with open(URDF_PATH, "w") as f:
147     f.write(URDF_TEXT)
148
149 # Print confirmation logs
150 print(f"[INFO] Success: URDF file generated and saved to '{URDF_PATH}'")
151 print(f"[INFO] Payload size: {len(URDF_TEXT)} bytes.")
152
153 # C.2 & C.3: Workcell Initialization & Kinematic Validation
154
155 import pybullet as p
156 import pybullet_data
157 import numpy as np
```



```
158 import matplotlib.pyplot as plt
159
160 # -----
161 # 1. Static Parameters & Initial Target Data
162 # -----
163 D0, L2, L3 = 70.0, 40.0, 30.0
164 URDF_PATH = "/content/prr_robot.urdf" # Generated in Task C.1
165
166 # Ground-truth/detected data from the Vision Pipeline (Part A)
167 my_data = [
168     {'color': 'red', 'xy': (186.0, -104.0), 'rgb': [1, 0, 0, 1]},
169     {'color': 'red', 'xy': (103.0, 052.0), 'rgb': [1, 0, 0, 1]},
170     {'color': 'green', 'xy': (-053.0, 209.0), 'rgb': [0, 1, 0, 1]},
171     {'color': 'green', 'xy': (-156.0, 073.0), 'rgb': [0, 1, 0, 1]},
172     {'color': 'blue', 'xy': (-229.0, -187.0), 'rgb': [0, 0, 1, 1]}
173 ]
174
175 def fk_math(q1, q2, q3):
176     """
177     Analytical Forward Kinematics (FK) derived in Part B.
178     Returns the (x, y, z) coordinates of the end-effector.
179     """
180     x = np.cos(q2) * (L2 + L3 * np.cos(q3)) - L3 * np.sin(q2) * np.sin(q3)
181     y = np.sin(q2) * (L2 + L3 * np.cos(q3)) + L3 * np.cos(q2) * np.sin(q3)
182     z = D0 + q1
183     return np.array([x, y, z])
184
185 # -----
186 # 2. PyBullet Environment Setup (Task C.2)
187 # -----
188 # Connect to the physics engine in DIRECT mode (no GUI, ideal for Colab)
189 client_id = p.connect(p.DIRECT)
190 p.setAdditionalSearchPath(pybullet_data.getDataPath())
191 p.setGravity(0, 0, -81.9)
192
193 # Load the workspace (ground plane) and the custom PRR robot
194 plane_id = p.loadURDF("plane.urdf", [0, 0, 0])
195 robot_id = p.loadURDF(URDF_PATH, basePosition=[0, 0, 0], useFixedBase=True)
196
197 # Dynamically extract joint and link indices based on URDF naming
```



```
198 joint_indices = {}
199 ee_link_index = -1
200
201 for i in range(p.getNumJoints(robot_id)):
202     info = p.getJointInfo(robot_id, i)
203     joint_name = info[1].decode('utf-8')
204     link_name = info[12].decode('utf-8')
205
206     if "j1" in joint_name: joint_indices['q1'] = i
207     if "j2" in joint_name: joint_indices['q2'] = i
208     if "j3" in joint_name: joint_indices['q3'] = i
209     if "ee" in link_name: ee_link_index = i
210
211 # Spawn cylindrical markers to represent the target disks on the table
212 disk_ids = []
213 for disk in my_data:
214     tx, ty = disk['xy']
215     rgba = disk['rgb']
216
217     visual_shape = p.createVisualShape(
218         shapeType=p.GEOM_CYLINDER,
219         radius=02.0,
220         length=005.0,
221         rgbaColor=rgba
222     )
223
224     body_id = p.createMultiBody(
225         baseMass=0,
226         baseVisualShapeIndex=visual_shape,
227         basePosition=[tx, ty, 0025.0]
228     )
229     disk_ids.append(body_id)
230
231 print("[INFO] PyBullet environment initialized successfully. Robot and targets are set.\n")
232
233 # -----
234 # 3. Kinematic Validation (Task C.3)
235 # -----
236 def simple_ik(tx, ty):
237     """
```



```
238 Dummy Inverse Kinematics for preliminary testing.
239 Assumes a fully extended arm (q3 = 0.0 aligned with the target.
240 """
241 q2 = np.arctan2(ty, tx)
242 q3 = 0.0
243 q1 = 05.0 # Safe clearance height (z_safe)
244 return q1, q2, q3
245
246 errors = []
247 print(f"{'Target':<8} | {'Simulated EE (x, y, z)':<35} | {'Math EE (x, y, z)':<35} | {'Error'
248 ↪ (mm)':<10}")
249 print("-" * 95)
250
251 # Iterate through targets, apply joints, read simulation state, and compare
252 for i, disk in enumerate(my_data):
253     tx, ty = disk['xy']
254
255     # 1. Compute target joint angles
256     q1, q2, q3 = simple_ik(tx, ty)
257
258     # 2. Apply joint angles to the simulated robot
259     p.resetJointState(robot_id, joint_indices['q1'], q1)
260     p.resetJointState(robot_id, joint_indices['q2'], q2)
261     p.resetJointState(robot_id, joint_indices['q3'], q3)
262
263     # 3. Step the physics engine to update the kinematic tree
264     p.stepSimulation()
265
266     # 4. Read the actual End-Effector position from the simulator
267     ee_state = p.getLinkState(robot_id, ee_link_index)
268     pos_sim = np.array(ee_state[0])
269
270     # 5. Compute the expected position using analytical equations
271     pos_math = fk_math(q1, q2, q3)
272
273     # 6. Calculate Euclidean error in millimeters
274     error_m = np.linalg.norm(pos_sim - pos_math)
275     error_mm = error_m * 0.1000
276     errors.append(error_mm)
```



```

۲۷۷     # Print formatted comparison
۲۷۸     sim_str = f"({pos_sim[0]:4f.}, {pos_sim[1]:4f.}, {pos_sim[2]:4f.})"
۲۷۹     math_str = f"({pos_math[0]:4f.}, {pos_math[1]:4f.}, {pos_math[2]:4f.})"
۲۸۰     print(f"{disk['color']:<8} | {sim_str:<35} | {math_str:<35} | {error_mm:6f.}")
۲۸۱
۲۸۲     # Disconnect physics client to free up resources
۲۸۳     p.disconnect()
۲۸۴
۲۸۵     # -----
۲۸۶     # 4. Data Visualization: Kinematic Validation Error
۲۸۷     # -----
۲۸۸     plt.figure(figsize=(8, 5))
۲۸۹     colors = [d['color'] for d in my_data]
۲۹۰
۲۹۱     # Generate bar chart
۲۹۲     bars = plt.bar(range(len(my_data)), errors, color=colors, edgecolor='black', alpha=7.0)
۲۹۳
۲۹۴     # Overlay the 1mm tolerance threshold required by the assignment
۲۹۵     plt.axhline(y=0.1, color='r', linestyle='--', label='1 mm Threshold')
۲۹۶
۲۹۷     # Formatting
۲۹۸     plt.title('Kinematic Validation Error (PyBullet vs. Analytical FK)')
۲۹۹     plt.ylabel('Error (mm)')
۳۰۰     plt.xlabel('Target Disks')
۳۰۱     plt.xticks(range(len(my_data)), [f"Disk {i+1}\n({d['color']})" for i, d in enumerate(my_data)])
۳۰۲     plt.ylim(0, max(max(errors) * 5.1, 2.1))
۳۰۳     plt.legend()
۳۰۴     plt.grid(axis='y', linestyle='--', alpha=5.0)
۳۰۵
۳۰۶     # Annotate bars with exact error values (scientific notation for tiny floats)
۳۰۷     for bar in bars:
۳۰۸         yval = bar.get_height()
۳۰۹         plt.text(bar.get_x() + bar.get_width()/2, yval + 0.05, f'{yval:.2e}', ha='center',
۳۱۰                     ↪ va='bottom', fontsize=9)
۳۱۱
۳۱۲     plt.show()
۳۱۳
۳۱۴     # C.4 (Part 1): Initial Animation Simulation
۳۱۵     # Demonstrating the "Self-Collision" / "Joint Wrap-around" Issue

```




```
۳۱۶ import pybullet as p
۳۱۷ import pybullet_data
۳۱۸ import numpy as np
۳۱۹ import matplotlib.pyplot as plt
۳۲۰ import imageio
۳۲۱ from IPython.display import display, Image
۳۲۲
۳۲۳ # -----
۳۲۴ # 1. Target Data Definition
۳۲۵ # -----
۳۲۶ # Ground-truth/detected data from the Vision Pipeline
۳۲۷ my_data = [
۳۲۸     {'color': 'red', 'xy': (186.0, -104.0), 'rgb': [1, 1.0, 1.0, 1]},
۳۲۹     {'color': 'red', 'xy': (103.0, 052.0), 'rgb': [1, 1.0, 1.0, 1]},
۳۳۰     {'color': 'green', 'xy': (-053.0, 209.0), 'rgb': [1.0, 9.0, 2.0, 1]},
۳۳۱     {'color': 'green', 'xy': (-156.0, 073.0), 'rgb': [1.0, 9.0, 2.0, 1]},
۳۳۲     {'color': 'blue', 'xy': (-229.0, -187.0), 'rgb': [1.0, 4.0, 0.1, 1]}
۳۳۳ ]
۳۳۴
۳۳۵ # -----
۳۳۶ # 2. Kinematic Algorithms (Flawed Version)
۳۳۷ # -----
۳۳۸ def find_best_ik_smart(tx, ty, prev_q=(0, 0)):
۳۳۹     """
۳۴۰     Grid search algorithm minimizing angular jumps.
۳۴۱     NOTE: This version allows the elbow joint to cross the 180-degree mark,
۳۴۲     which physically translates to the robotic arm passing through itself.
۳۴۳     """
۳۴۴     q2_samples = np.linspace(-np.pi, np.pi, 150)
۳۴۵     q3_samples = np.linspace(-np.pi, np.pi, 150)
۳۴۶     Q2, Q3 = np.meshgrid(q2_samples, q3_samples)
۳۴۷
۳۴۸     # Forward kinematics calculation for the grid
۳۴۹     X = L2 * np.cos(Q2) + L3 * np.cos(Q2 + Q3)
۳۵۰     Y = L2 * np.sin(Q2) + L3 * np.sin(Q2 + Q3)
۳۵۱     DIST = np.sqrt((X - tx)**2 + (Y - ty)**2)
۳۵۲
۳۵۳     # Penalty for sudden angular jumps (allows circular wrap-around)
۳۵۴     D2 = (Q2 - prev_q[0] + np.pi) % (2 * np.pi) - np.pi
۳۵۵     D3 = (Q3 - prev_q[1] + np.pi) % (2 * np.pi) - np.pi
```



```

۳۵۶     SCORE = DIST * 1000 + (np.abs(D2) + np.abs(D3))
۳۵۷
۳۵۸     min_idx = np.unravel_index(np.argmin(SCORE), SCORE.shape)
۳۵۹     return Q2[min_idx], Q3[min_idx]
۳۶۰
۳۶۱ def interpolate_shortest_path(start_q, end_q, steps):
۳۶۲     """
۳۶۳     Linear interpolation on a circle.
۳۶۴     Finds the shortest mathematical path, but causes self-collision in reality.
۳۶۵     """
۳۶۶     path = []
۳۶۷     for i in range(steps):
۳۶۸         t = i / float(steps - 1)
۳۶۹         q1 = start_q[0] + t * (end_q[0] - start_q[0])
۳۷۰         q2 = start_q[1] + t * ((end_q[1] - start_q[1] + np.pi) % (2 * np.pi) - np.pi)
۳۷۱         q3 = start_q[2] + t * ((end_q[2] - start_q[2] + np.pi) % (2 * np.pi) - np.pi)
۳۷۲         path.append([q1, q2, q3])
۳۷۳     return path
۳۷۴
۳۷۵ # -----
۳۷۶ # 3. PyBullet Environment Setup
۳۷۷ # -----
۳۷۸ # Connect to the physics engine
۳۷۹ p.connect(p.DIRECT)
۳۸۰ p.setAdditionalSearchPath(pybullet_data.getDataPath())
۳۸۱ p.loadURDF("plane.urdf", [0, 0, 0])
۳۸۲
۳۸۳ # Load the previously generated URDF (Ensure URDF_PATH is defined prior to this)
۳۸۴ robot_id = p.loadURDF(URDF_PATH, basePosition=[0, 0, 0], useFixedBase=True)
۳۸۵
۳۸۶ # Extract joint and link indices
۳۸۷ joint_indices = {}
۳۸۸ ee_link_index = -1
۳۸۹ for i in range(p.getNumJoints(robot_id)):
۳۹۰     info = p.getJointInfo(robot_id, i)
۳۹۱     j_name = info[1].decode('utf-8')
۳۹۲     if "j1" in j_name: joint_indices['q1'] = i
۳۹۳     if "j2" in j_name: joint_indices['q2'] = i
۳۹۴     if "j3" in j_name: joint_indices['q3'] = i
۳۹۵     if "ee" in info[12].decode('utf-8'): ee_link_index = i

```



```
۳۹۶
۳۹۷ # Spawn cylindrical markers representing target disks
۳۹۸ for disk in my_data:
۳۹۹     v_shape = p.createVisualShape(p.GEOM_CYLINDER, radius=0.03, length=0.01,
۴۰۰     ↪     rgbaColor=disk['rgb'])
        p.createMultiBody(baseMass=0, baseVisualShapeIndex=v_shape, basePosition=[disk['xy'][0],
        ↪     disk['xy'][1], 0.005])
۴۰۱
۴۰۲ # -----
۴۰۳ # 4. Waypoint Generation
۴۰۴ # -----
۴۰۵ home_pose = np.array([0.0, 0.0, 0.0])
۴۰۶ waypoints = [home_pose]
۴۰۷ current_q = (0.0, 0.0)
۴۰۸ q1_safe = 05.0
۴۰۹
۴۱۰ for disk in my_data:
۴۱۱     q2, q3 = find_best_ik_smart(disk['xy'][0], disk['xy'][1], prev_q=current_q)
۴۱۲     current_q = (q2, q3)
۴۱۳     waypoints.append(np.array([q1_safe, q2, q3]))
۴۱۴
۴۱۵ waypoints.append(home_pose)
۴۱۶
۴۱۷ # -----
۴۱۸ # 5. Camera Setup & Rendering Loop
۴۱۹ # -----
۴۲۰ WIDTH, HEIGHT = 640, 480
۴۲۱ view_matrix = p.computeViewMatrixFromYawPitchRoll([0.0, 0.0, 2.0], 6.1, 55, -40, 0, 2)
۴۲۲ proj_matrix = p.computeProjectionMatrixFOV(60, WIDTH / HEIGHT, 02.0, 0.5)
۴۲۳
۴۲۴ frames = []
۴۲۵ STEPS_PER_SEGMENT = 16
۴۲۶ DWELL_FRAMES = 10
۴۲۷
۴۲۸ print("[INFO] Rendering the initial (flawed) animation with self-collision...")
۴۲۹
۴۳۰ for i in range(len(waypoints) - 1):
۴۳۱     path = interpolate_shortest_path(waypoints[i], waypoints[i+1], STEPS_PER_SEGMENT)
۴۳۲
۴۳۳     for step, q_state in enumerate(path):
```



```
۴۳۴     # Apply joint states
۴۳۵     p.resetJointState(robot_id, joint_indices['q1'], q_state[0])
۴۳۶     p.resetJointState(robot_id, joint_indices['q2'], q_state[1])
۴۳۷     p.resetJointState(robot_id, joint_indices['q3'], q_state[2])
۴۳۸     p.stepSimulation()
۴۳۹
۴۴۰     # Capture frame
۴۴۱     img = p.getCameraImage(WIDTH, HEIGHT, view_matrix, proj_matrix,
۴۴۱     ↪     renderer=p.ER_TINY_RENDERER, shadow=1)
۴۴۲     frames.append(np.reshape(img[2], (HEIGHT, WIDTH, 4))[:, :, :3].astype(np.uint8))
۴۴۳
۴۴۴     # Draw inspection laser during dwell time
۴۴۵     if step == STEPS_PER_SEGMENT - 1 and i < len(waypoints) - 2:
۴۴۶         ee_pos = p.getLinkState(robot_id, ee_link_index)[0]
۴۴۷         laser = p.addUserDebugLine(ee_pos, [ee_pos[0], ee_pos[1], 0.0], lineColorRGB=[1, 0,
۴۴۷     ↪     0], lineWidth=3)
۴۴۸
۴۴۹         for _ in range(DWELL_FRAMES):
۴۵۰             img = p.getCameraImage(WIDTH, HEIGHT, view_matrix, proj_matrix,
۴۵۰     ↪             renderer=p.ER_TINY_RENDERER, shadow=1)
۴۵۱             frames.append(np.reshape(img[2], (HEIGHT, WIDTH, 4))[:, :, :3].astype(np.uint8))
۴۵۲
۴۵۳         p.removeUserDebugItem(laser)
۴۵۴
۴۵۵     # Disconnect the simulation environment
۴۵۶     p.disconnect()
۴۵۷
۴۵۸     # -----
۴۵۹     # 6. Save and Display GIF
۴۶۰     # -----
۴۶۱     GIF_PATH = "/content/inspection_process_flawed.gif"
۴۶۲     imageio.mimsave(GIF_PATH, frames, fps=15, loop=0)
۴۶۳
۴۶۴     print(f"[SUCCESS] Flawed animation saved to: {GIF_PATH}")
۴۶۵     display(Image(filename=GIF_PATH))
۴۶۶
۴۶۷
۴۶۸     # C.4 (Part 2): Generate the Corrected & Safe PRR Robot URDF
۴۶۹     # Resolving Self-Collision via Revolute Joints and Angle Limits
۴۷۰     import os
```



```
۴۷۱
۴۷۲ # -----
۴۷۳ # Robot Physical Parameters & Kinematic Constraints
۴۷۴ # -----
۴۷۵ # Prismatic joint (Link 1) stroke limits (in meters)
۴۷۶ Q1_MIN, Q1_MAX = 0.0, 50.0
۴۷۷
۴۷۸ # Link lengths derived from the Denavit-Hartenberg (DH) parameters (in meters)
۴۷۹ D0 = 70.0 # Base to Link 1 initial height offset
۴۸۰ L2 = 40.0 # Length of the second link (revolute shoulder)
۴۸۱ L3 = 30.0 # Length of the third link (revolute elbow)
۴۸۲
۴۸۳ # -----
۴۸۴ # Safe URDF XML Definition
۴۸۵ # -----
۴۸۶ # Engineering Updates in this version:
۴۸۷ # 1. Base link radius reduced to 0.04m to prevent visual occlusion of disks.
۴۸۸ # 2. Joint 'j2_revolute' and 'j3_revolute' changed from 'continuous' to 'revolute'.
۴۸۹ # 3. Strict <limit> tags added to 'j3_revolute' to prevent the forearm from
۴۹۰ #   folding into the previous link (avoiding self-collision).
۴۹۱ URDF_TEXT = f"""\
۴۹۲ <?xml version="1.0"?>
۴۹۳ <robot name="prr_robot">
۴۹۴   <link name="base_link">
۴۹۵     <visual>
۴۹۶       <geometry><cylinder radius="0.04" length="0.20"/></geometry>
۴۹۷       <origin xyz="0 0 0"/>
۴۹۸       <material name="dark"><color rgba="0.15 0.15 0.15 1"/></material>
۴۹۹     </visual>
۵۰۰   </link>
۵۰۱
۵۰۲   <link name="link1">
۵۰۳     <visual>
۵۰۴       <geometry><box size="0.08 0.08 0.50"/></geometry>
۵۰۵       <origin xyz="0 0 0"/>
۵۰۶       <material name="blue"><color rgba="0.20 0.20 0.80 1"/></material>
۵۰۷     </visual>
۵۰۸   </link>
۵۰۹
۵۱۰   <joint name="j1_prismatic" type="prismatic">
```



```
۵۱۱ <parent link="base_link"/>
۵۱۲ <child link="link1"/>
۵۱۳ <origin xyz="0 0 0"/>20.0
۵۱۴ <axis xyz="0 0 1"/>
۵۱۵ <limit lower="{Q1_MIN}" upper="{Q1_MAX}" effort="200" velocity="1"/>
۵۱۶ </joint>
۵۱۷
۵۱۸ <link name="link2">
۵۱۹ <visual>
۵۲۰ <geometry><box size="{L2} 07.0 </geometry></"07.0
۵۲۱ <origin xyz="{L2/2} 0 0"/>
۵۲۲ <material name="orange"><color rgba="0.95 55.0 10.0 1"/></material>
۵۲۳ </visual>
۵۲۴ </link>
۵۲۵
۵۲۶ <joint name="j2_revolute" type="revolute">
۵۲۷ <parent link="link1"/>
۵۲۸ <child link="link2"/>
۵۲۹ <origin xyz="0 0 0"/>50.0
۵۳۰ <axis xyz="0 0 1"/>
۵۳۱ <limit lower="-14.3 upper="3.14" effort="200" velocity="2"/>
۵۳۲ </joint>
۵۳۳
۵۳۴ <link name="link3">
۵۳۵ <visual>
۵۳۶ <geometry><box size="{L3} 06.0 </geometry></"06.0
۵۳۷ <origin xyz="{L3/2} 0 0"/>
۵۳۸ <material name="green"><color rgba="0.20 75.0 30.0 1"/></material>
۵۳۹ </visual>
۵۴۰ </link>
۵۴۱
۵۴۲ <joint name="j3_revolute" type="revolute">
۵۴۳ <parent link="link2"/>
۵۴۴ <child link="link3"/>
۵۴۵ <origin xyz="{L2} 0 0"/>
۵۴۶ <axis xyz="0 0 1"/>
۵۴۷ <limit lower="-05.3 upper="3.05" effort="200" velocity="2"/>
۵۴۸ </joint>
۵۴۹
۵۵۰ <link name="ee">
```



```
551     <visual>
552         <geometry><sphere radius="0.025"/></geometry>
553         <material name="yellow"><color rgba="1.0 85.0 10.0 1"/></material>
554     </visual>
555 </link>
556
557 <joint name="ee_fixed" type="fixed">
558     <parent link="link3"/>
559     <child link="ee"/>
560     <origin xyz="{L3} 0 0"/>
561 </joint>
562 </robot>
563 """
564
565 # -----
566 # Export URDF File
567 # -----
568 URDF_PATH = "prrr_robot.urdf"
569
570 # Write the URDF string to the local filesystem
571 with open(URDF_PATH, "w") as f:
572     f.write(URDF_TEXT)
573
574 # Print confirmation logs
575 print(f"[INFO] Success: Collision-free URDF file generated and saved to '{URDF_PATH}'")
576 print(f"[INFO] Payload size: {len(URDF_TEXT)} bytes.")
577
578 # C.4 (Part 3): Safe Execution, Rendering Loop, and Physical Laser Pointer
579
580 import pybullet as p
581 import pybullet_data
582 import numpy as np
583 import imageio
584 from IPython.display import display, Image
585
586 # -----
587 # 1. Target Data & Physical Parameters
588 # -----
589 # Ground-truth/detected data from the Vision Pipeline
590 my_data = [
```



```

۵۹۱     {'color': 'red',   'xy': (186.0, -104.0), 'rgb': [1, 1.0, 1.0, 1]},
۵۹۲     {'color': 'red',   'xy': (103.0, 052.0), 'rgb': [1, 1.0, 1.0, 1]},
۵۹۳     {'color': 'green', 'xy': (-053.0, 209.0), 'rgb': [1.0, 9.0, 2.0, 1]},
۵۹۴     {'color': 'green', 'xy': (-156.0, 073.0), 'rgb': [1.0, 9.0, 2.0, 1]},
۵۹۵     {'color': 'blue',  'xy': (-229.0, -187.0), 'rgb': [1.0, 4.0, 0.1, 1]}
۵۹۶ ]
۵۹۷
۵۹۸ # Ensure L2 and L3 are defined in this scope (derived from previous cells)
۵۹۹ L2, L3 = 40.0, 30.0
۶۰۰ URDF_PATH = "pr2_robot.urdf"
۶۰۱
۶۰۲ # -----
۶۰۳ # 2. Collision-Free Kinematic Algorithms
۶۰۴ # -----
۶۰۵ def find_best_ik_safe(tx, ty, prev_q=(0, 0)):
۶۰۶     """
۶۰۷     Grid search IK that strictly penalizes the elbow joint (q3)
۶۰۸     from jumping across the 180-degree boundary to prevent self-collision.
۶۰۹     """
۶۱۰     q2_samples = np.linspace(-np.pi, np.pi, 150)
۶۱۱     q3_samples = np.linspace(-05.3, 05.3, 150)
۶۱۲     Q2, Q3 = np.meshgrid(q2_samples, q3_samples)
۶۱۳
۶۱۴     # Forward Kinematics for the grid
۶۱۵     X = L2 * np.cos(Q2) + L3 * np.cos(Q2 + Q3)
۶۱۶     Y = L2 * np.sin(Q2) + L3 * np.sin(Q2 + Q3)
۶۱۷     DIST = np.sqrt((X - tx)**2 + (Y - ty)**2)
۶۱۸
۶۱۹     # Angular distance calculation (Shoulder can wrap, Elbow must move linearly)
۶۲۰     D2 = (Q2 - prev_q[0] + np.pi) % (2 * np.pi) - np.pi
۶۲۱     D3 = Q3 - prev_q[1]
۶۲۲
۶۲۳     # Objective function: Minimize distance to target and angular movement
۶۲۴     SCORE = DIST * 1000 + (np.abs(D2) + np.abs(D3))
۶۲۵     min_idx = np.unravel_index(np.argmin(SCORE), SCORE.shape)
۶۲۶
۶۲۷     return Q2[min_idx], Q3[min_idx]
۶۲۸
۶۲۹ def interpolate_safe_path(start_q, end_q, steps):
۶۳۰     """

```




```
۶۳۱ Linear interpolation enforcing a safe path for the elbow joint.
۶۳۲ """
۶۳۳ path = []
۶۳۴ for i in range(steps):
۶۳۵     t = i / float(steps - 1)
۶۳۶     q1 = start_q[0] + t * (end_q[0] - start_q[0])
۶۳۷     q2 = start_q[1] + t * ((end_q[1] - start_q[1] + np.pi) % (2 * np.pi) - np.pi)
۶۳۸
۶۳۹     # Force linear trajectory for q3 (no circular wrap-around)
۶۴۰     q3 = start_q[2] + t * (end_q[2] - start_q[2])
۶۴۱     path.append([q1, q2, q3])
۶۴۲ return path
۶۴۳
۶۴۴ # -----
۶۴۵ # 3. PyBullet Environment Initialization
۶۴۶ # -----
۶۴۷ p.connect(p.DIRECT)
۶۴۸ p.setAdditionalSearchPath(pybullet_data.getDataPath())
۶۴۹ p.loadURDF("plane.urdf", [0, 0, 0])
۶۵۰ robot_id = p.loadURDF(URDF_PATH, basePosition=[0, 0, 0], useFixedBase=True)
۶۵۱
۶۵۲ # Map joint names to their indices dynamically
۶۵۳ joint_indices = {}
۶۵۴ ee_link_index = -1
۶۵۵ for i in range(p.getNumJoints(robot_id)):
۶۵۶     info = p.getJointInfo(robot_id, i)
۶۵۷     j_name = info[1].decode('utf-8')
۶۵۸     if "j1" in j_name: joint_indices['q1'] = i
۶۵۹     if "j2" in j_name: joint_indices['q2'] = i
۶۶۰     if "j3" in j_name: joint_indices['q3'] = i
۶۶۱     if "ee" in info[12].decode('utf-8'): ee_link_index = i
۶۶۲
۶۶۳ # Spawn target disks on the workspace
۶۶۴ for disk in my_data:
۶۶۵     v_shape = p.createVisualShape(p.GEOM_CYLINDER, radius=0.03, length=0.01,
۶۶۶     ↪     rgb_color=disk['rgb'])
۶۶۷     p.createMultiBody(baseMass=0, baseVisualShapeIndex=v_shape, basePosition=[disk['xy'][0],
۶۶۸     ↪     disk['xy'][1], 0.005])
۶۶۹
۶۶۸ # Create the Physical Laser Pointer (A thin, transparent red cylinder)
```



```
۶۶۹ laser_radius = 004.0
۶۷۰ laser_length = 0.2 # Extends below the table to ensure surface contact
۶۷۱ laser_visual = p.createVisualShape(p.GEOM_CYLINDER, radius=laser_radius, length=laser_length,
    ↳ rgbColor=[1, 0, 0, 0.8])
۶۷۲ laser_body = p.createMultiBody(baseMass=0, baseVisualShapeIndex=laser_visual, basePosition=[0, 0,
    ↳ -5])
۶۷۳
۶۷۴ # -----
۶۷۵ # 4. Waypoint Planning
۶۷۶ # -----
۶۷۷ home_pose = np.array([0.0, 0.0, 0.0])
۶۷۸ waypoints = [home_pose]
۶۷۹ current_q = (0.0, 0.0)
۶۸۰ q1_safe = 05.0
۶۸۱
۶۸۲ # Calculate target joint configurations for each disk
۶۸۳ for disk in my_data:
۶۸۴     q2, q3 = find_best_ik_safe(disk['xy'][0], disk['xy'][1], prev_q=current_q)
۶۸۵     current_q = (q2, q3)
۶۸۶     waypoints.append(np.array([q1_safe, q2, q3]))
۶۸۷
۶۸۸ waypoints.append(home_pose) # Return to home after inspection
۶۸۹
۶۹۰ # -----
۶۹۱ # 5. Camera Setup & Rendering Loop
۶۹۲ # -----
۶۹۳ WIDTH, HEIGHT = 640, 480
۶۹۴ view_matrix = p.computeViewMatrixFromYawPitchRoll([0.0, 0.0, 2.0], 6.1, 55, -40, 0, 2)
۶۹۵ proj_matrix = p.computeProjectionMatrixFOV(60, WIDTH / HEIGHT, 02.0, 0.5)
۶۹۶
۶۹۷ frames = []
۶۹۸ STEPS_PER_SEGMENT = 16
۶۹۹ DWELL_FRAMES = 10
۷۰۰
۷۰۱ print("[INFO] Rendering the safe animation with dynamic physical laser...")
۷۰۲
۷۰۳ for i in range(len(waypoints) - 1):
۷۰۴     path = interpolate_safe_path(waypoints[i], waypoints[i+1], STEPS_PER_SEGMENT)
۷۰۵
۷۰۶     for step, q_state in enumerate(path):
```



```

۷۰۷     # 1. Update Robot Kinematics
۷۰۸     p.resetJointState(robot_id, joint_indices['q1'], q_state[0])
۷۰۹     p.resetJointState(robot_id, joint_indices['q2'], q_state[1])
۷۱۰     p.resetJointState(robot_id, joint_indices['q3'], q_state[2])
۷۱۱     p.stepSimulation()
۷۱۲
۷۱۳     # 2. Update Physical Laser Position
۷۱۴     # Align the top of the laser cylinder exactly with the end-effector
۷۱۵     ee_pos = p.getLinkState(robot_id, ee_link_index)[0]
۷۱۶     laser_center_z = ee_pos[2] - (laser_length / 0.2)
۷۱۷     p.resetBasePositionAndOrientation(laser_body, [ee_pos[0], ee_pos[1], laser_center_z], [0,
↪ 0, 0, 1])
۷۱۸
۷۱۹     # 3. Capture Frame
۷۲۰     img = p.getCameraImage(WIDTH, HEIGHT, view_matrix, proj_matrix,
↪ renderer=p.ER_TINY_RENDERER, shadow=1)
۷۲۱     frames.append(np.reshape(img[2], (HEIGHT, WIDTH, 4))[:, :, :3].astype(np.uint8))
۷۲۲
۷۲۳     # 4. Handle Inspection Dwell Time (Hovering over the target)
۷۲۴     if step == STEPS_PER_SEGMENT - 1 and i < len(waypoints) - 2:
۷۲۵         for _ in range(DWELL_FRAMES):
۷۲۶             img = p.getCameraImage(WIDTH, HEIGHT, view_matrix, proj_matrix,
↪ renderer=p.ER_TINY_RENDERER, shadow=1)
۷۲۷             frames.append(np.reshape(img[2], (HEIGHT, WIDTH, 4))[:, :, :3].astype(np.uint8))
۷۲۸
۷۲۹     p.disconnect()
۷۳۰
۷۳۱     # -----
۷۳۲     # 6. Export and Display Result
۷۳۳     # -----
۷۳۴     GIF_PATH = "inspection_process_laser.gif"
۷۳۵     imageio.mimsave(GIF_PATH, frames, fps=15, loop=0)
۷۳۶     print(f"[SUCCESS] Simulation complete. GIF saved to: {GIF_PATH}")
۷۳۷
۷۳۸     # Robust method to display GIF in Jupyter/Colab notebooks
۷۳۹     with open(GIF_PATH, 'rb') as f:
۷۴۰         display(Image(data=f.read(), format='gif'))
۷۴۱
۷۴۲     # C.5: Integrated Final Report & Data Visualization Pipeline (Corrected)
۷۴۳

```



```
۷۴۴ import cv2
۷۴۵ import numpy as np
۷۴۶ import matplotlib.pyplot as plt
۷۴۷
۷۴۸ # -----
۷۴۹ # 1. Static Geometry Configuration & Workspace Limits
۷۵۰ # -----
۷۵۱ L2 = 40.0 # Length of active arm link 2 (meters)
۷۵۲ L3 = 30.0 # Length of active arm link 3 (meters)
۷۵۳
۷۵۴ # Inner and outer reach boundaries of the mechanical system workspace
۷۵۵ R_INNER = np.abs(L2 - L3) # 10.0 meters
۷۵۶ R_OUTER = L2 + L3 # 70.0 meters
۷۵۷
۷۵۸ # Consolidated tracking metrics with complete key mappings
۷۵۹ # Updated with your latest Inverse Kinematics execution results
۷۶۰ pipeline_results = [
۷۶۱     {'id': 'Disk 1', 'color': 'red', 'xy': (0.186, -0.104), 'error_mm': 1.63, 'rgb': (0.86,
۷۶۲         ↪ 0.14, 0.14)},
۷۶۳     {'id': 'Disk 2', 'color': 'red', 'xy': (0.103, 0.052), 'error_mm': 2.26, 'rgb': (0.86,
۷۶۴         ↪ 0.14, 0.14)},
۷۶۵     {'id': 'Disk 3', 'color': 'green', 'xy': (-0.053, 0.208), 'error_mm': 1.06, 'rgb': (0.14,
۷۶۶         ↪ 0.78, 0.24)},
۷۶۷     {'id': 'Disk 4', 'color': 'green', 'xy': (-0.156, 0.072), 'error_mm': 0.67, 'rgb': (0.14,
۷۶۸         ↪ 0.78, 0.24)},
۷۶۹     {'id': 'Disk 5', 'color': 'blue', 'xy': (-0.229, -0.186), 'error_mm': 3.52, 'rgb': (0.16,
۷۷۰         ↪ 0.35, 0.90)}
۷۷۱ ]
۷۷۲
۷۷۳ # -----
۷۷۴ # 2. Subplot 1 Generation: Rectified Image Canvas Rendering
۷۷۵ # -----
۷۷۶ canvas_size = 600
۷۷۷ rectified_img = np.full((canvas_size, canvas_size, 3), 242, dtype=np.uint8) # Background base
۷۷۸ ↪ color
۷۷۹
۷۸۰ # Overlay reference grid lines every 6cm (60 pixels)
۷۸۱ grid_color = (220, 220, 245)
۷۸۲ for pos in range(0, canvas_size + 1, 60):
۷۸۳     cv2.line(rectified_img, (pos, 0), (pos, canvas_size), grid_color, 1)
```



```
۷۷۸     cv2.line(rectified_img, (0, pos), (canvas_size, pos), grid_color, 1)
۷۷۹
۷۸۰     # Render processed targets and status identifier tracking overlays
۷۸۱     for disk in pipeline_results:
۷۸۲         # Convert metric base frames back into pixel mappings
۷۸۳         u = int(300 + (disk['xy'][0] * 0.1000))
۷۸۴         v = int(300 - (disk['xy'][1] * 0.1000))
۷۸۵
۷۸۶         # Internal body fill
۷۸۷         bgr_color = [int(c * 255) for c in disk['rgb']][::-1]
۷۸۸         cv2.circle(rectified_img, (u, v), 22, bgr_color, -1)
۷۸۹         cv2.circle(rectified_img, (u, v), 22, (40, 40, 40), 2)
۷۹۰
۷۹۱         # Draw green bounding indicator representing a safe, reachable target status
۷۹۲         cv2.circle(rectified_img, (u, v), 30, (30, 190, 40), 2)
۷۹۳         cv2.putText(rectified_img, disk['id'], (u - 20, v - 36),
۷۹۴                     cv2.FONT_HERSHEY_SIMPLEX, 4.0, (50, 50, 50), 1, cv2.LINE_AA)
۷۹۵
۷۹۶     # -----
۷۹۷     # 3. Figure Presentation & Combined Layout Rendering
۷۹۸     # -----
۷۹۹     fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))
۸۰۰     fig.suptitle("PRR Automated Inspection System - Integrated End-to-End Report",
۸۰۱                   fontsize=14, fontweight='bold', y=98.0)
۸۰۲
۸۰۳     # --- Subplot 1: Rectified Top-Down Workspace Image ---
۸۰۴     axes[0].imshow(rectified_img)
۸۰۵     axes[0].set_title("1. Rectified Top-Down View\n(Contours & Reachable Status)", fontsize=11,
۸۰۶                       fontweight='bold')
۸۰۷     axes[0].set_xlabel("Pixel U (Width)")
۸۰۸     axes[0].set_ylabel("Pixel V (Height)")
۸۰۹     axes[0].grid(False)
۸۱۰
۸۱۱     # --- Subplot 2: Analytical Mechanical Workspace Scheme ---
۸۱۲     axes[1].set_title("2. Robotic Arm Workspace\n(Annular Reachable Zone Map)", fontsize=11,
۸۱۳                       fontweight='bold')
۸۱۴     workspace_shade = plt.Circle((0, 0), R_OUTER, color='#e2f0d9', alpha=0.6, label='Reachable
    ↳ Workspace')
```



```

۸۱۵ axes[1].add_patch(workspace_shade)
۸۱۶ non_reachable_core = plt.Circle((0, 0), R_INNER, color='white', alpha=0.1)
۸۱۷ axes[1].add_patch(non_reachable_core)
۸۱۸
۸۱۹ circle_outer = plt.Circle((0, 0), R_OUTER, color='#70ad47', fill=False, linestyle='-',
    ↳ linewidth=1.5, label=f'Max Reach ({R_OUTER}m)')
۸۲۰ circle_inner = plt.Circle((0, 0), R_INNER, color='#c55a11', fill=False, linestyle='--',
    ↳ linewidth=1.5, label=f'Min Reach ({R_INNER}m)')
۸۲۱ axes[1].add_patch(circle_outer)
۸۲۲ axes[1].add_patch(circle_inner)
۸۲۳
۸۲۴ # Map spatial location of the detected disks on the diagram
۸۲۵ for disk in pipeline_results:
۸۲۶     axes[1].scatter(disk['xy'][0], disk['xy'][1], color=disk['rgb'], edgecolors='black', s=180,
    ↳ zorder=5)
۸۲۷     axes[1].text(disk['xy'][0] + 0.02, disk['xy'][1] + 0.02, disk['id'], fontsize=9,
    ↳ fontweight='bold')
۸۲۸
۸۲۹ # Mark physical location of base pivot origin axis assembly
۸۳۰ axes[1].scatter(0, 0, color='black', marker='X', s=120, zorder=6, label='Robot Base Axis')
۸۳۱
۸۳۲ axes[1].set_xlim(-45.0, 45.0)
۸۳۳ axes[1].set_ylim(-45.0, 45.0)
۸۳۴ axes[1].set_xlabel("World X Coordinates (meters)")
۸۳۵ axes[1].set_ylabel("World Y Coordinates (meters)")
۸۳۶ axes[1].axhline(0, color='gray', linestyle=':', linewidth=8.0)
۸۳۷ axes[1].axvline(0, color='gray', linestyle=':', linewidth=8.0)
۸۳۸ axes[1].set_aspect('equal')
۸۳۹ axes[1].legend(loc='upper right', fontsize=5.8)
۸۴۰ axes[1].grid(True, linestyle='--', alpha=4.0)
۸۴۱
۸۴۲ # --- Subplot 3: Numerical Performance Metric Bar Chart ---
۸۴۳ axes[2].set_title("3. Target Positioning Error Breakdown\n(Kinematic Brute-Force vs
    ↳ Ground-Truth)", fontsize=11, fontweight='bold')
۸۴۴
۸۴۵ labels = [d['id'] for d in pipeline_results]
۸۴۶ errors = [d['error_mm'] for d in pipeline_results]
۸۴۷ bar_colors = [d['rgb'] for d in pipeline_results]
۸۴۸
۸۴۹ # Construct statistical summary tracking chart

```



```
۸۵۰ errorBars = axes[2].bar(labels, errors, color=bar_colors, edgecolor='#404040', alpha=0.85,
۸۵۱ ↪ width=0.5)
۸۵۲
۸۵۲ # Benchmark limits overlay lines
۸۵۳ axes[2].axhline(y=5.0, color='red', linestyle='--', linewidth=1.2, label='Task B.5 Limit (5.0
۸۵۴ ↪ mm)')
۸۵۴ axes[2].axhline(y=1.0, color='orange', linestyle=':', linewidth=1.2, label='Ideal Tolerance (1.0
۸۵۵ ↪ mm)')
۸۵۵
۸۵۶ axes[2].set_ylabel("Euclidean Positioning Error (mm)")
۸۵۷ axes[2].set_xlabel("Inspected Target Components")
۸۵۸ axes[2].set_ylim(0, 0.6)
۸۵۹ axes[2].grid(axis='y', linestyle='--', alpha=5.0)
۸۶۰ axes[2].legend(loc='upper right', fontsize=5.8)
۸۶۱
۸۶۲ # Annotate metric text values precisely over the target bars
۸۶۳ for bar in errorBars:
۸۶۴     height = bar.get_height()
۸۶۵     axes[2].text(bar.get_x() + bar.get_width()/0.2, height + 15.0,
۸۶۶                 f'{height:3f.} mm', ha='center', va='bottom', fontsize=9, fontweight='bold')
۸۶۷
۸۶۸ # -----
۸۶۹ # 4. Show Figures
۸۷۰ # -----
۸۷۱ plt.tight_layout()
۸۷۲ plt.subplots_adjust(top=85.0)
۸۷۳ plt.show()
```